University of Sussex

School of Engineering and Informatics

G5038: Individual Project

Final Report

AI-Powered Dynamic Narrative System for RPGs

Author: Duncan Law

Candidate Number: 281967

Degree Course: BSc Digital Media and Games Computing

Supervisor: Dr Dan Creed

Year of Submission: 2026

Word Count: 13412

Word count excludes: bibliography, footnotes/endnotes, appendices, abstract/summary, figure legends, tables, and acknowledgements, per University of Sussex assessment policy.

University of Sussex
Engineering & Informatics

Statement of Originality

This report is submitted as part requirement for the degree of BSc Digital Media and Games Computing at the University of Sussex. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

I hereby **give /** withhold permission for a copy of this report to be loaned out to students in future years. *(delete as necessary)*

Signature: _____

Date: _____

Acknowledgements

I would like to thank my supervisor, Dr Dan Creed, for his guidance and support throughout this project. His feedback during our regular meetings helped shape both the technical direction and the critical analysis presented in this report.

On a personal note, I am deeply grateful to the late Sita Chan. Although she tragically passed away in 2013, her enduring music provided the essential soundtrack that kept me focused and comforted during countless late-night coding and debugging sessions.

Finally, this project has been a personal reminder of the importance of balance. The process of building a game that critiques relentless productivity and blind optimism naturally prompted reflection on those same values in my own approach to work. I hope that message resonates with anyone who plays it.

Summary

This project investigates the integration of Large Language Models (LLMs) into interactive narrative games through the development of "Glorious Deliverance Agency 1," a 2D text-based RPG built in the Godot engine. The game employs a bespoke pipeline architecture where the LLM operates within explicit developer-defined constraints to generate dynamic narrative content whilst maintaining thematic coherence.

The implementation features a multi-provider AI architecture supporting Gemini, OpenRouter, Ollama, and Mock providers, enabling development flexibility and cost management. A three-layer context management system balances narrative coherence with token efficiency, whilst the "Reality vs. Positive Energy" thematic framework constrains AI generation to produce satirically appropriate content.

Key technical contributions include the Strategy pattern for provider abstraction, an EventBus for cross-module communication, and defensive multi-stage response parsing to handle non-deterministic AI outputs. The project demonstrates that bespoke pipeline approaches can successfully integrate LLM-generated content into playable games, providing practical insights for developers considering similar integrations.

Functional testing across approximately one hundred narrative generations achieved fewer than ten percent requiring intervention due to tonal inconsistency, meeting the defined success criteria. The comparative analysis identifies concrete trade-offs between bespoke pipelines and world model approaches regarding control versus emergence, development complexity versus scope, and predictability versus novelty.

Contents

Statement of Originality **1**

Acknowledgements **2**

Summary **3**

1 Introduction **1**
- 1.1 Project Aims and Objectives 1
- 1.2 Justification of Technical and Narrative Choices 2
- 1.3 Target User Group and Scope 3
- 1.4 Expected Findings and Achievements 3
- 1.5 Report Structure 3

2 Background and Literature Review **4**
- 2.1 AI in Games and Generative Narrative Systems 4
- 2.2 Large Language Models for Interactive Narratives 5
- 2.3 Bespoke Pipelines vs. World-Model Approaches 5
- 2.4 Coherence and Evaluation in AI-Generated Narratives 6
- 2.5 Runtime Constraints and Practical Considerations 6
- 2.6 Thematic Control and Values-Based Systems 7
- 2.7 Research Gaps and Implications for My Project 8
- 2.8 Context Management in AI Development Tools 9

3 Professional Considerations and Ethical Framework **9**
- 3.1 Primary Risk: AI Content Generation 9
- 3.2 Functional Testing as Ethical Safeguard 10
- 3.3 Control Measures 10
- 3.4 Data Protection 10

4 Project Specification and Methodology **11**

5 System Design **13**

6 Project Management Reflection **19**
- 6.1 Planning Approach and Its Impact 20
- 6.2 Challenges and Adaptations 20
- 6.3 Lessons for Future Projects 20

7 Design **21**

7.1 Architectural Philosophy: Separation of Deterministic and Non-Deterministic Systems . . . 21
7.2 Design Patterns and Their Justifications . . . 22
7.3 Context Management Architecture . . . 22
7.4 Prompt Engineering Architecture . . . 23
7.5 Response Parsing and Validation . . . 23
7.6 Safety and Content Filtering . . . 23
7.7 Design Evolution During Implementation . . . 24

8 Implementation . . . 24
8.1 The AI Integration Journey: From Simple Calls to Robust Architecture . . . 24
8.2 Prompt Engineering: An Iterative Learning Process . . . 25
8.3 Response Parsing Challenges . . . 26
8.4 Memory Management: Balancing Context and Token Cost . . . 26
8.5 The Butterfly Effect System . . . 26
8.6 Error Handling and Graceful Degradation . . . 27
8.7 Key Code Examples . . . 27

9 Testing . . . 29
9.1 Testing Strategy Overview . . . 29
9.2 Unit Testing Approach . . . 30
9.3 Integration Testing Between Components . . . 30
9.4 Functional Testing for AI Behaviour . . . 30
9.5 Bugs Discovered and Resolved . . . 31
9.6 Test Coverage and Limitations . . . 32

10 Evaluation . . . 32
10.1 Technical Functionality Assessment . . . 32
10.2 Thematic Coherence Assessment . . . 33
10.3 Comparative Analysis: Bespoke Pipeline vs. World Model . . . 33
10.3.1 Understanding World Models . . . 34
10.3.2 Architectural Trade-offs . . . 35
10.3.3 The Shifting Role of the Developer . . . 36
10.3.4 Practical Feasibility Assessment . . . 37
10.3.5 Implications for Future Development . . . 39
10.4 Player Experience Evaluation . . . 40
10.5 Original Requirements Assessment . . . 41
10.6 Limitations and Honest Assessment . . . 43

11 Conclusion . . . 44

11.1 Summary of Achievements 44
11.2 Key Lessons Learned 44
11.3 Future Work 45
11.4 Personal Reflection 46
11.5 Final Statement 46

Bibliography **47**

A Ethics Compliance Form **51**

B Project Proposal **52**

C Game Design Document **53**

1 Introduction

1.1 Project Aims and Objectives

I am developing "Glorious Deliverance Agency 1," a 2D text-based RPG built in the Godot engine that uses Large Language Model (LLM) integration to generate dynamic narrative content. The game places players as a reluctant hero in a dysfunctional team, whose attempts to save the world with "positive energy" only accelerate its destruction through a darkly comedic narrative that responds to their choices in real-time.

Traditional RPG narratives rely on pre-scripted dialogue trees and fixed storylines, limiting replayability and player agency. While LLMs offer the potential for dynamic content generation, integrating them into games presents significant challenges: maintaining narrative coherence over extended play sessions, ensuring thematic consistency with authorial intent, and managing the practical constraints of API latency and token costs. I investigated whether a carefully structured "bespoke pipeline" approach—where the LLM operates within explicit developer-defined constraints—can address these challenges to produce a playable, thematically coherent RPG experience. My central research question is therefore: to what extent can a text-based LLM, integrated as a controlled component within a traditional game engine, generate coherent and thematically consistent narrative content for an interactive RPG?

My primary aim is supported by three foundational pillars. The first pillar, Technical Development, involved the design and implementation of a functional AI-driven 2D RPG narrative system. The game features dynamically generated storylines and NPC dialogue, where "dynamically" refers to the real-time generation of narrative content based on the current game state and player choices, moving beyond traditional pre-scripted narratives. My technical focus was on enabling emergent, non-linear story paths while maintaining narrative coherence over extended gameplay sessions through a three-layer context management system (detailed in Appendix C: GDD, Section 7.2).

The second pillar, Thematic Execution, centres on creating a satirical allegory exploring the conflict between "positive energy" ideology and the concept of Void Entropy. The core gameplay mechanics, including the Reality vs. Positive Energy system and the Prayer System, deliberately subvert conventional RPG expectations (see Appendix C: GDD, Sections 4.2–4.3 for detailed mechanics). I have embedded these thematic concepts directly into the AI's prompt engineering, ensuring that generated content aligns with my project's darkly satirical tone rather than producing generic positive outcomes.

The third pillar, Critical Analysis, involved producing a comparative analysis of my "bespoke pipeline" methodology against the theoretical "world model" paradigm. I examined architectural trade-offs between fine-grained developer control and emergent simulation capabilities,

explored the shifting role of the developer from systems architect to high-level director, and assessed the practical feasibility of each approach based on my direct implementation experience.

To ensure these aims remain achievable within the project's constraints, I have defined concrete success criteria. For Technical Development, success means producing a functional prototype that can generate coherent narrative content across multiple consecutive scenes without critical errors, with measurable metrics including API response latency under five seconds and successful context preservation across save/load cycles. For Thematic Execution, success means the AI consistently generates content that aligns with the "Reality vs. Positive Energy" framework during functional testing, with fewer than one in ten generations requiring manual intervention due to tonal inconsistency. For Critical Analysis, success means producing a structured comparison document that draws on my direct implementation experience to identify at least three concrete architectural trade-offs between bespoke pipelines and world models. These criteria are deliberately scoped to be demonstrable through functional testing rather than requiring extensive user studies.

1.2 Justification of Technical and Narrative Choices

I selected the Godot 4.x engine as my development platform due to its open-source nature and scene-node architecture, which facilitates the construction of UI-heavy applications suitable for narrative-driven games. The engine's support for both C# and GDScript provides flexibility for integrating external AI services through custom modules, while its lightweight footprint ensures rapid iteration during development.

I employed Gemini-2.5-flash as my primary LLM provider, selected based on latency and cost performance, convenient API and toolchain integration, and sufficient context length to support extended interactive narrative sessions (Google, 2025). Actual available context and performance vary by model and configuration; therefore, my final evaluation criteria are based on functional test results within my project's specific scenario, including long-term narrative consistency, response time, and cost monitoring, rather than pure specification comparisons. This approach ensures my assessment reflects practical implementation requirements rather than theoretical capabilities.

My narrative framework, centred on the thematic tension between "Reality" and "Positive Energy," serves a dual purpose. First, it provides a testbed for evaluating the AI's capacity to maintain thematic coherence within a defined conceptual space. Second, the deliberately ironic and darkly satirical tone presents a challenge for the model's stylistic consistency, testing whether the system can generate content that aligns with specific tonal and thematic constraints rather than producing generic narrative outputs. I designed this approach to explore how effectively a bespoke pipeline can maintain authorial intent within AI-generated content.

1.3 Target User Group and Scope

During development, I served as the primary user, functioning as the sole tester in a functional testing capacity. This reflects my project's nature as a research prototype investigating AI-driven narrative generation, where the unpredictable nature of LLM outputs necessitates thorough internal validation before external exposure.

The intended eventual audience for "Glorious Deliverance Agency 1" is detailed in Appendix C: Game Design Document, Section 1.5 (Target Audience), which defines both primary and secondary user groups. The game's dark satirical tone and thematic content make it unsuitable for younger audiences; a mature content advisory would be appropriate for any public release.

For the purposes of this academic project, however, I limited the scope to demonstrating technical feasibility and thematic coherence through developer-led functional testing. I did not plan formal user studies with external participants at this stage, as the ethical framework I outline in Section 3 required first establishing confidence in the system's output behaviour.

1.4 Expected Findings and Achievements

By the project's completion, I demonstrated that a bespoke pipeline architecture can successfully integrate LLM-generated content into a playable RPG while maintaining thematic coherence. Specifically, I showed that my three-layer context management system preserves narrative consistency across extended play sessions, that the "Reality vs. Positive Energy" framework effectively constrains AI generation to produce satirically appropriate content, and that the multi-provider architecture enables practical development within budget constraints. The comparative analysis identified concrete trade-offs between bespoke pipelines and world model approaches, providing insights for developers considering similar integrations. While the prototype did not achieve commercial-quality polish, it successfully demonstrates the viability of controlled LLM integration for narrative games.

1.5 Report Structure

This report is organised as follows. Section 2 reviews relevant literature on AI in games, LLM integration, and narrative coherence, establishing the theoretical foundation for my approach. Section 3 addresses professional and ethical considerations, particularly regarding AI-generated content. Section 4 presents my project specification and methodology, including success criteria and evaluation plans. Section 5 provides an overview of the system design through architectural diagrams. Section 6 reflects on project management, discussing planning approaches and lessons learned. Section 7 details the design decisions and patterns employed in my implementation. Section 8 describes the implementation process, including challenges encountered and solutions developed. Section 9 covers my testing strategy and results. Section 10 evaluates the

project against original requirements and provides a comparative analysis of bespoke pipelines versus world models. Finally, Section 11 concludes with a summary of achievements, lessons learned, and directions for future work.

2 Background and Literature Review

2.1 AI in Games and Generative Narrative Systems

When I began this project, I assumed game AI meant NPCs following behaviour trees or finite state machines—reactive systems that respond to player actions but cannot create anything genuinely new (Riedl, 2021). Reading the literature changed how I understood what AI in games could be. The emerging field of generative narrative systems, increasingly powered by Procedural Content Generation (PCG) and Large Language Models (LLMs), treats story not as a fixed script but as a co-creative space shaped by both the player and the system in real time (Sun et al., 2023; Al-Surayhi et al., 2025). This was the paradigm I wanted to build towards: in "Glorious Deliverance Agency 1," I positioned the LLM not as a character within the story but as the "story director" itself, orchestrating scene transitions, generating contextual NPC responses, and determining narrative consequences based on the player's accumulated choices and current game state.

Reading about early approaches to automated story generation—story grammars, planning algorithms—helped me understand why I needed something more flexible (AlHussain and Azmi, 2021). The concept that most directly shaped my architecture was the "storylet": a small, self-contained narrative unit with defined prerequisites and effects that the system sequences dynamically based on game state (Short, 2019). I adopted this pattern in my AIManager, which generates discrete narrative "beats" triggered by specific game conditions—player stats, previous choices, current location—each producing defined effects on the game state. Reading Short's work convinced me that modular, condition-driven generation was more tractable than attempting to produce continuous storylines.

The challenge I kept returning to in the literature was the "controllability problem": how do you let the AI generate freely enough to surprise the player, whilst constraining it tightly enough to maintain thematic coherence (Balint and Bidarra, 2023; AlHussain and Azmi, 2021)? This question became the central engineering problem of my project. I addressed it through a multi-layered context model in my AIManager: a static "world bible" layer establishing firm thematic guardrails—the rules of "Glorious Deliverance Agency 1," the nature of "Positive Energy" corruption, the characteristics of each NPC—within which the LLM generates content guided by short-term event memory and a summarised long-term history. Whether this hybrid architecture could produce a narrative that was coherent, thematically resonant, and dynamically responsive to player choice was the question my entire implementation set out to answer.

2.2 Large Language Models for Interactive Narratives

What struck me most when reading the LLM literature was how quickly the field had moved from theoretical to practical. Researchers had already demonstrated that LLMs could generate context-aware NPC dialogue aligned with character personas (Chen, 2025), and produce procedural quest descriptions comparable to human-authored content (Värtinen et al., 2022). These results came from prompt engineering and fine-tuning rather than from re-training models—a finding that shaped my entire approach. I applied these techniques directly: my AIManager constructs prompts that embed persona definitions for each NPC in "Glorious Deliverance Agency 1," so that Saint Gloria's toxic positivity, Sir Donkey's delusional heroism, and ARK's obsessive bureaucracy each maintain distinct voices across interactions (Yang et al., 2024).

The architectural dichotomy I found most relevant was the distinction between the "world model" approach and the "bespoke pipeline" approach. World models—exemplified by Park et al.'s (2023) "Generative Agents"—aim to simulate entire environments and populations from the ground up, with agents that possess memory, can reflect, and plan across long time horizons. The bespoke pipeline, by contrast, integrates an LLM as one component within a traditional game engine, grounding it with structured game state data and constraining its output to specific formats (Góngora et al., 2025). Reading these two approaches side by side clarified my own position: in "Glorious Deliverance Agency 1," the Godot engine maintains authoritative game state—player stats, inventory, relationship values, narrative flags—whilst the LLM operates purely as a content generation service, receiving that state as context and returning narrative text.

Understanding this trade-off early was important. World models promise unprecedented emergence and player freedom, but the literature was honest about their limitations: ensuring a consistent, meaningful story in a fully emergent simulation remains an open problem, and the computational cost is immense (Yang et al., 2024). Bespoke pipelines offer authorial control by design, as the LLM's scope is constrained by the game's pre-defined structure, but that control comes at the cost of emergent possibility (Góngora et al., 2025). Wan et al. (2025) reinforced something I would later discover for myself—that making LLMs adhere to complex constraints, even in controlled pipelines, is non-trivial and requires continuous architectural attention rather than a one-time configuration.

2.3 Bespoke Pipelines vs. World-Model Approaches

Here I contextualise my architectural choice before the implementation chapters that follow. A bespoke pipeline integrates a general-purpose LLM as a component within a traditional game engine—in my case, Godot plus an LLM API—enabling controlled state management and narrative guidance through explicit developer-defined constraints. When I began researching alternatives, I found that world model approaches pursue end-to-end generation of entire interactive

environments, but current publicly accessible systems remain insufficient for a single-developer project of this scope (Parker-Holder and Fruchter, 2025). The bespoke pipeline was therefore not merely a preference but a practical necessity. My implementation reflects this clearly: the Godot engine handles all game logic, state persistence, and UI rendering, whilst the LLM API is invoked only for narrative text generation, with clearly defined input schemas and output formats. This separation allowed me to develop, test, and debug each layer independently.

2.4 Coherence and Evaluation in AI-Generated Narratives

Narrative coherence was the problem I worried about most before starting implementation. The literature confirmed my concern: coherence—the logical and causal consistency of events, character behaviours, and world stability across a play session—is precisely where generative systems tend to fail (Yi et al., 2025). In "Glorious Deliverance Agency 1," coherence failures would manifest as NPCs forgetting previous conversations, the game contradicting established plot points, or the satirical tone shifting unexpectedly to sincere positivity.

Reading this literature also taught me something important about evaluation. Recent work proposes automated benchmarking frameworks for narrative coherence (Wan and Ma, 2025; El Boudouri et al., 2025), but for a single-developer project generating non-deterministic outputs, I concluded that functional testing—direct observation and developer assessment—was more appropriate than quantitative scoring. I adopted a protocol built around three questions: did the AI maintain the "Reality vs Positive Energy" thematic framework? Did generated content align with the established dark satirical tone? Did the context management system prevent contradictions across extended play sessions? My specific test scenarios included playing through multiple sessions with the same save state, deliberately making contradictory choices, and monitoring whether NPC dialogue correctly referenced previous player actions stored in the context memory.

The memory management literature directly informed how I solved long-horizon coherence. Research shows that systems using summarisation and retrieval can provide LLMs with sufficient long-term context to avoid contradictions without exceeding context windows (Yi et al., 2025; Park et al., 2023). I implemented this through my three-layer context model, with specific token allocations and layer specifications detailed in Appendix C: GDD, Section 7.2.

2.5 Runtime Constraints and Practical Considerations

Latency was a constraint I underestimated before I started building. The literature was clear that even a few hundred milliseconds of delay can break immersion in interactive narrative, making an AI-driven character feel unresponsive (Schomay, 2024; Juliani, 2025). For a live-service game at scale, this would require highly optimised smaller models or significant investment in edge computing infrastructure to bring inference closer to the player (English, 2025). I

addressed this through the Strategy pattern in my AIManager, which allowed me to switch between providers at runtime depending on performance conditions—something I documented in Appendix C: GDD, Section 7.1.

Token budgets were the constraint I thought about most during implementation. Every API call costs tokens, and reading about how production systems handle this taught me techniques I would not have thought of independently. Rather than feeding entire conversation histories into each prompt, the industry uses hierarchical summarisation, Retrieval-Augmented Generation to pull in only relevant context, and automated pruning of stale data—approaches demonstrated by platforms like Anthropic's Claude and GitHub Copilot (Sridi, 2025; Anthropic, 2025; DZone, 2024). I implemented token monitoring directly in my AIManager, logging input and output counts per API call to track cumulative costs and identify where I could optimise.

The content safety literature raised a concern I had not fully anticipated: hallucination. LLMs can generate content that deviates from the established narrative or character persona in ways that are subtle and hard to detect automatically (O'Brien, 2024). Reading about the PANGeA framework—which uses the LLM itself to validate player inputs against game rules and generate in-character corrective responses (Buongiorno et al., 2024)—gave me a concrete model for how to think about guardrails. My solution was simpler but more fundamental: I constrained the input space at the UI level, requiring players to choose from predefined buttons rather than type freely. This single design decision eliminated the largest category of unexpected prompt inputs before they could reach the AI at all.

Finally, Juliani's (2025) warning about "infinite trash"—the risk that unconstrained LLM generation produces content that is technically coherent but thematically meaningless—stayed with me throughout development. It helped me understand why I needed not just content constraints but directional ones: the AI had to know not only what was off-limits, but what the narrative was actively trying to achieve. A related finding from Vidler and Walsh (2025) reinforced this: LLMs exhibit strong statistical biases even in simple game-like scenarios such as Rock-Paper-Scissors, meaning that without explicit directional constraints, the AI's apparent "choice" is not truly random but shaped by distributional patterns in its training data. For a narrative game built around ironic inversion, this bias could easily work against my satirical intent if left unaddressed.

2.6 Thematic Control and Values-Based Systems

One problem I had not anticipated when designing my satirical framework was what Juliani (2025) calls "mode collapse" in LLM generation: models prompted for a specific genre tend to over-rely on its most statistically common surface markers, producing repetitive and shallow content. A model asked for a "JRPG narrative" gravitates towards words like "whispers" and "echoes" because they dominate that genre's training data. For my project's dark satire, this

was a genuine risk—I needed the AI to produce a very specific tone, not a statistical average of dark fantasy tropes.

Reading about this problem led me to design an explicit "values-based system" rather than relying on genre prompting alone (see Appendix C: GDD, Section 4.2 for detailed formulas). Every player choice has a quantifiable impact on the "Reality" and "Positive Energy" stats, which feed directly back into the AI's prompts to modulate tone and direction. This transforms the game's central theme from a static premise into a measurable, directive principle—the AI is not merely told to be satirical; it is given a numerical measure of how far the world has collapsed, and instructed to generate content that reflects that specific state.

This design also resolved the "infinite trash" problem (Juliani, 2025) in a way I found elegant. Rather than aiming for complete narrative freedom, I framed player agency within a core ironic loop: positive intentions are systematically converted into negative outcomes. The Prayer System embodies this approach (see Appendix C: GDD, Section 4.3). Player freedom is preserved, but its consequences are strictly channelled through the game's thematic lens, striking a balance between dynamic generation and authorial control.

2.7 Research Gaps and Implications for My Project

Having surveyed the literature, I identified a gap that my project could directly address. Research either examines theoretical world models at scale (Parker-Holder and Fruchter, 2025) or general LLM integration without tight thematic constraints (OpenAI, 2023; Pichai and Hassabis, 2023). What is missing is a practical account of what it actually costs—in engineering effort, in prompt complexity, in architectural decisions—to maintain specific author-driven thematic control within a bespoke pipeline. That gap is what this project fills.

My original contributions emerged from trying to solve concrete problems I encountered. The three-layer context architecture (static world bible, short-term event memory, summarised long-term history) grew out of the memory management literature and the practical need to stay within token budgets whilst preserving narrative continuity. The values-based generation control system—where quantified game stats directly modify prompt parameters to enforce thematic alignment—came from reading about mode collapse and realising that implicit genre cues were insufficient. The functional testing framework I developed for evaluating thematic coherence in non-deterministic outputs arose from recognising that formalised benchmarks were not designed for single-developer projects working with a specific satirical voice. Finally, the comparative analysis between my bespoke pipeline experience and the world model literature contributes grounded empirical observations to a discourse that is otherwise largely speculative.

2.8 Context Management in AI Development Tools

Token management was a problem I had to solve before I could build anything reliable. Extended gameplay sessions exceeding thirty minutes risk exceeding model context windows, and I discovered early in testing that simply including more context did not always help—excessive input degraded output quality even when it fit within the limit. Reading about how GitHub Copilot and Anthropic's Claude handle this taught me that the solution is selective inclusion: relevance-based filtering, summarisation, and external memory systems rather than raw concatenation (DZone, 2024; Anthropic, 2025). I implemented these principles as a three-layer context architecture—static world rules, short-term event memory, and summarised long-term history—designed to balance narrative detail against token efficiency whilst maintaining coherence across extended play sessions.

This literature review directly shaped my architectural decisions. The storylet concept informed my modular narrative generation approach. The memory management research informed my three-layer context system. The mode collapse problem motivated my values-based prompt control. And the bespoke pipeline versus world model distinction gave me the conceptual framework to understand what I was building and why the constraints I imposed were necessary rather than merely convenient.

3 Professional Considerations and Ethical Framework

Building this project required me to consider professional and ethical responsibilities carefully, guided by the British Computer Society (BCS) Code of Conduct. Having studied relevant modules at the University of Sussex, I can claim competence in the technical areas this project addresses (BCS Code of Conduct 2.a, 2.b). All claims and referenced works are properly documented to avoid misrepresentation (BCS Code of Conduct 2.f).

3.1 Primary Risk: AI Content Generation

The key ethical insight for my project is that the primary risk is not traditional data privacy concerns, but rather the potential for generative AI to produce unexpected or inappropriate content. Given my project's deliberately dark satirical narrative framework, there exists a genuine possibility that the AI could generate content that crosses boundaries of acceptability. At this stage of development, it is not possible to predict with certainty what content the AI might generate, which is precisely why formalised user testing with external participants is not appropriate until the system's output behaviour has been thoroughly validated.

3.2 Functional Testing as Ethical Safeguard

I adopted functional testing as both a methodological approach and an ethical safeguard, serving as the sole tester to observe what content the AI generates and whether it remains within acceptable boundaries. This is an ongoing process, as the nondeterministic nature of LLM outputs means each generation session may produce different results.

The following categories constitute definitive red lines that would trigger immediate testing termination and require architectural intervention before resumption. Content depicting or encouraging real-world violence, self-harm, or suicide represents the first and most critical red line; while my game's satirical framework permits dark humour about abstract concepts like "entropy," any generation promoting genuine harm to individuals is unacceptable. The second red line concerns discriminatory content targeting protected characteristics; the satire must critique systems and behaviours rather than denigrate individuals based on race, gender, religion, disability, or other protected attributes. The third red line addresses sexually explicit material, which is entirely outside my project's thematic scope and would indicate a fundamental failure of prompt constraints. The fourth red line involves content that constitutes legal liability, such as defamatory statements about real individuals or institutions, or content that could reasonably be interpreted as incitement.

3.3 Control Measures

My primary mitigation strategy is embedded in the core UI design: the system uses predefined button choices for player interaction rather than open-ended text input, significantly constraining the space of possible prompts sent to the AI. I explicitly considered and rejected voice input due to ethical concerns including the potential for inadvertently recording background conversations.

The Prayer System permits short text input as an ironic game mechanic but remains constrained by cooldown timers, templated prompt construction, and content filtering.

3.4 Data Protection

API keys are stored exclusively on my local machine within Godot's sandboxed user directory, excluded from version control. System logs capture AI prompts and responses for evaluation purposes, stored locally without personally identifiable information. When local Ollama models are used, all processing occurs on local hardware with no third-party data transmission.

These ethical considerations shaped my technical decisions throughout the project, particularly the choice to use button-based input rather than free text and the implementation of content filtering.

Having established the ethical framework guiding this project, Section 4 translates these commitments into a formal project specification and methodology.

4 Project Specification and Methodology

I built the system using the Godot 4.x game engine with a modular architecture employing singleton manager classes to provide globally accessible services. The system architecture, component specifications, and interface definitions are fully documented in Appendix C: Game Design Document, Sections 7.1–7.6.

My system employs a Strategy design pattern supporting multiple provider types (detailed in Appendix C: GDD, Section 7.1). This architecture enables development flexibility and cost management through provider switching.

As I identified in Section 2.8, effective context management is critical for maintaining narrative coherence. I implemented a three-layer context model balancing narrative coherence with token efficiency (see Appendix C: GDD, Section 7.2 for complete specifications).

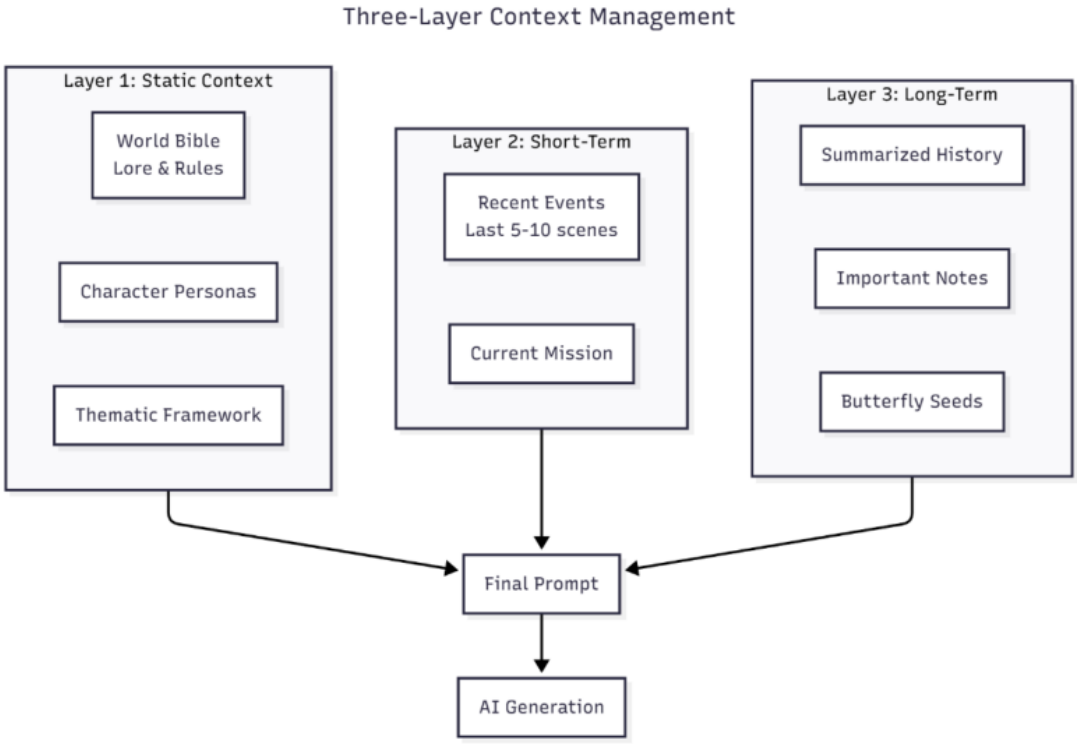


Figure 1: Three-Layer Context Management Architecture. The diagram illustrates how static context, short-term context, and long-term memory are combined into a final prompt for AI generation.

I evaluated the project through functional testing focused on determining whether the system performed as intended across three key dimensions: technical functionality, thematic coherence,

and player experience quality. The testing strategy and evidence plan are detailed in Appendix C: GDD, Section 5.

I defined success as achieving functional technical implementation where core systems operate reliably without critical errors, maintaining thematic coherence where AI-generated content consistently aligns with the established satirical framework, and demonstrating player experience potential where implemented mechanics exhibit qualities associated with engaging gameplay.

My evaluation assessed whether core systems operated correctly through direct testing. Key questions included: did the AI integration successfully generate narrative content? Did the provider switching mechanisms function reliably? Did the context management system maintain state across extended sessions without errors? Did the game handle AI generation failures gracefully? I instrumented the AIManager to log performance metrics, including API latency and token consumption, enabling analysis of cost-effectiveness and responsiveness trade-offs between different AI providers. Implementation details for the testing infrastructure are documented in Appendix C: GDD, Section 5.1–5.2.

Functional testing determined whether AI-generated content aligned with my project's specific thematic intent. Did the system maintain the "Reality vs. Positive Energy" thematic framework consistently? Did the Prayer System reliably produce ironic, satirically appropriate outcomes rather than generic positive responses? Did my three-layer context model effectively prevent narrative contradictions and thematic drift across extended gameplay? As the functional tester, I made qualitative assessments of whether generated content served the artistic vision or undermined it.

While formal player testing with external participants was not appropriate at this development stage, my functional testing process evaluated whether the system generated content that could constitute an engaging player experience. Drawing on established game design frameworks, "fun" in this context comprises several elements: challenge (do player choices present meaningful dilemmas?), rules (are the game's systems comprehensible and consistent?), goals (does the narrative create clear objectives and progression?), and options (does the AI provide sufficiently varied choices?). I assessed whether the implemented features—dynamic narrative generation, state-based choice consequences, teammate interference systems, moral dilemmas—functioned not merely technically but also served to create an experience with these qualities.

My comparison to "world model" approaches remained purely theoretical, based on public documentation (Parker-Holder and Fruchter, 2025), focusing on architectural trade-offs rather than direct performance benchmarks. I drew on my implementation experience and functional testing observations from this bespoke pipeline to provide grounded insights into the practical implications of architectural choices.

My methodology assumed that a mock AI provider is sufficient for most UI and logic development. I acknowledge the inherent variability and non-determinism of LLM outputs, which presents a challenge for repeatable testing. I operated under a limited API budget, reinforcing the need for the Ollama and mock provider paths.

The third pillar of my project, the critical analysis, was conducted by comparing my "bespoke pipeline" methodology against the theoretical "world model" paradigm through conceptual and architectural analysis based on publicly available research documentation (Parker-Holder and Fruchter, 2025). This is not an empirical benchmark, as world model technologies are not publicly available for direct testing.

I drew from two primary sources: (a) my direct implementation experience and evaluation data from this project, which serves as a concrete example of a bespoke pipeline, and (b) the publicly available technical reports, research papers, and blog posts describing world model research such as Google's Genie project (Parker-Holder and Fruchter, 2025) and the underlying capabilities of large models from providers like OpenAI (OpenAI, 2023).

My comparison framework contrasted: (1) architectural trade-offs between the modular bespoke pipeline (Godot Engine + LLM API) offering fine-grained control and integrated world models promising emergent scope; and (2) the shifting developer role from systems architect in bespoke approaches to high-level director in world model paradigms. This structured analysis provided grounded insights into the practical implications of architectural choices, drawing on my direct implementation experience.

Adopting this methodology proved essential for managing the inherent unpredictability of LLM-based systems. The spiral development approach allowed me to iteratively refine prompt engineering and context management based on observed AI behaviour, whilst the multi-provider strategy ensured development continuity despite external API constraints.

With the methodology established, Section 5 presents the system design that emerged from applying it.

5 System Design

I have designed the system architecture to be modular and scalable, built around decoupled singleton manager components and purpose-built UI scene controllers. This separation of concerns is critical for managing non-deterministic AI components. Components communicate through clearly defined signal-based interfaces to minimise coupling while enabling complex AI feedback loops. Comprehensive technical specifications are documented in Appendix C: Game Design Document, Sections 7.1–7.6. The following diagrams provide a high-level overview of the architecture.

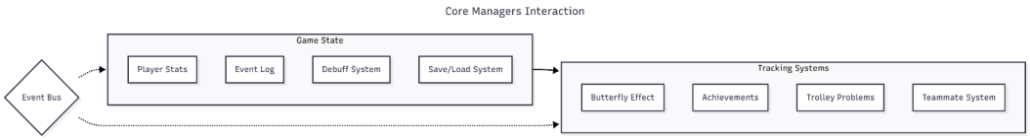


Figure 2: Core Managers Interaction. The Event Bus facilitates communication between the Game State subsystem (Player Stats, Event Log, Debuff System, Save/Load System) and the Tracking Systems (Butterfly Effect, Achievements, Trolley Problems, Teammate System).

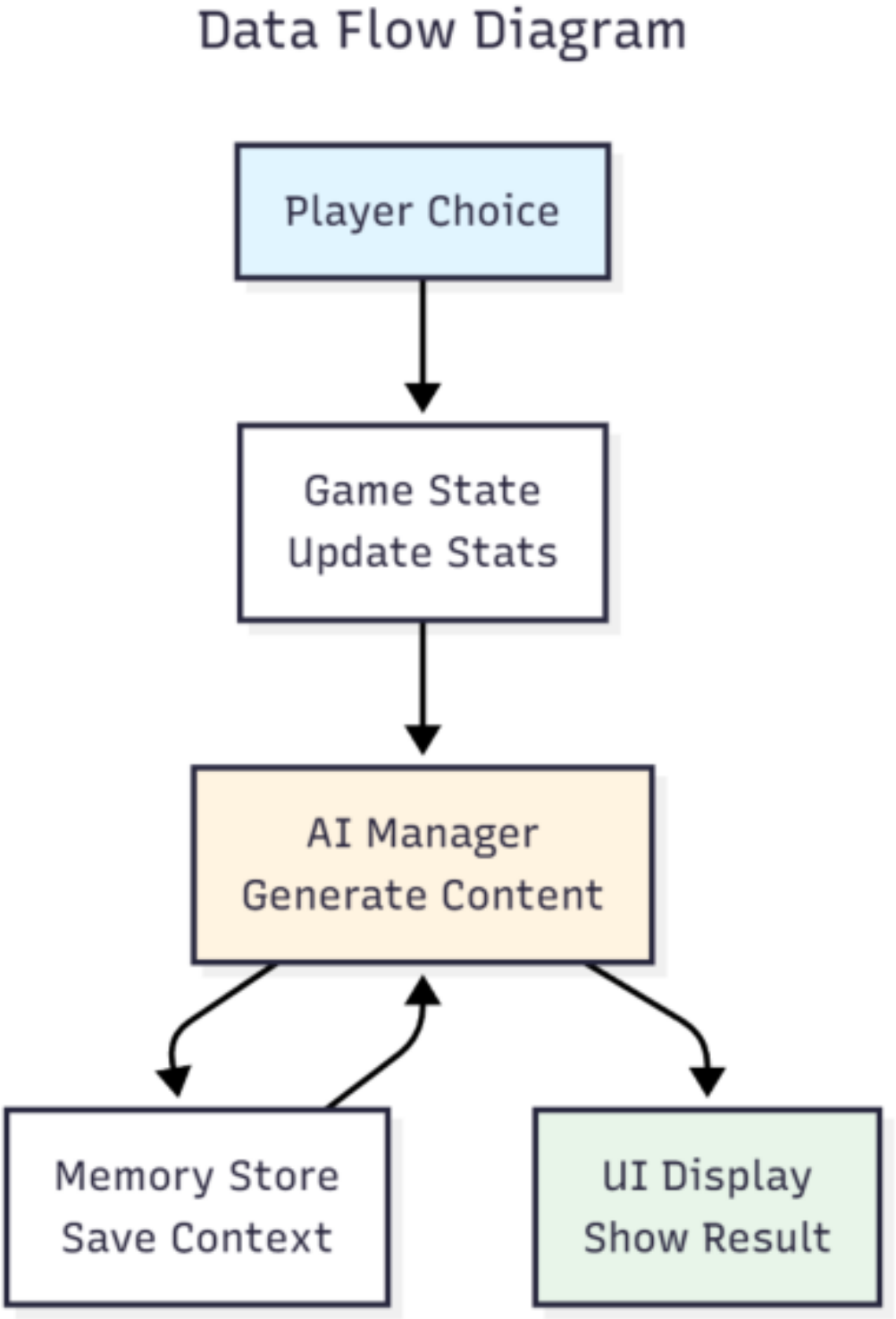
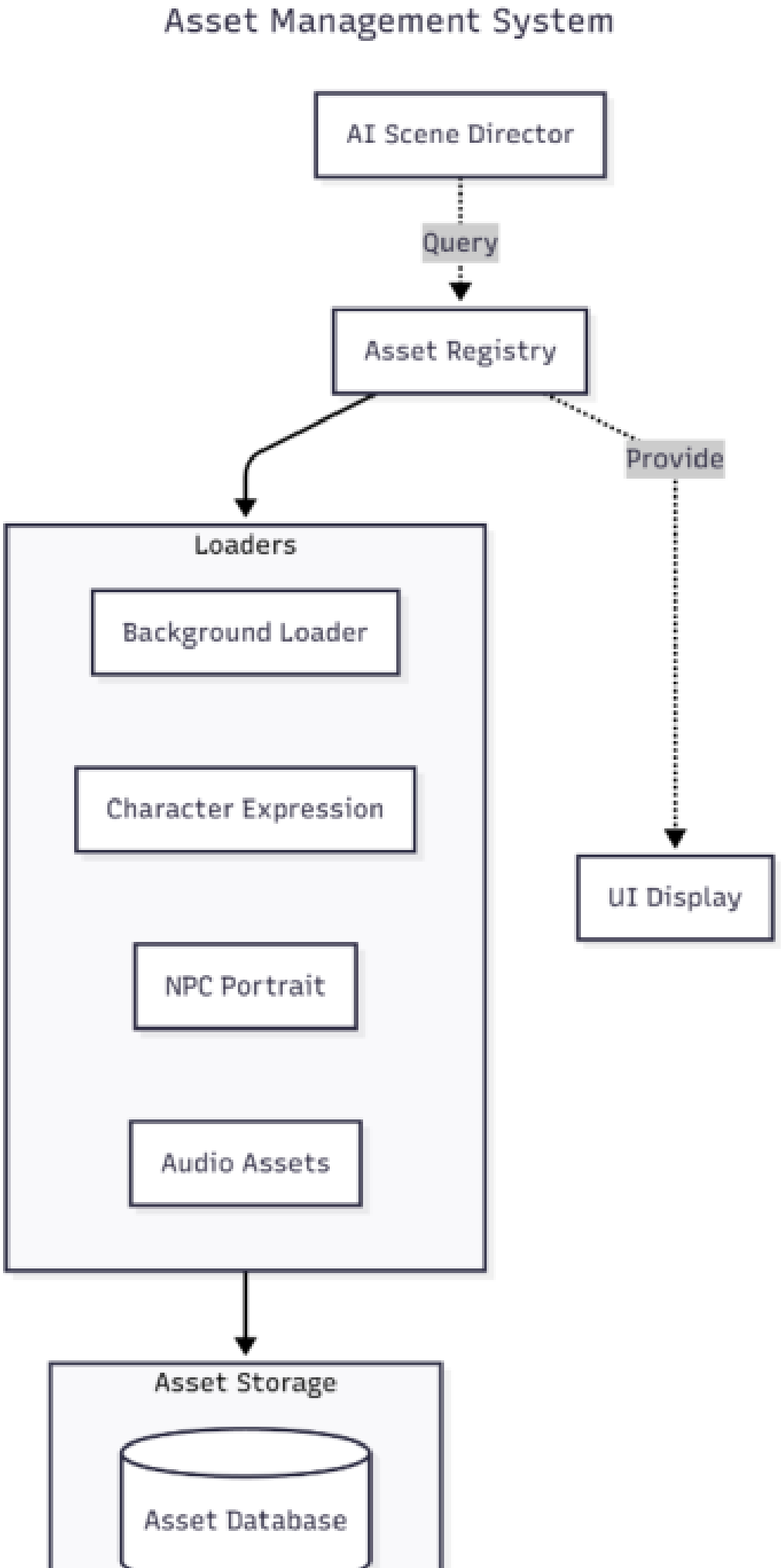


Figure 3: Data Flow Diagram. Player choices trigger game state updates, which inform the AI Manager's content generation. Generated content is simultaneously stored in the memory system and displayed through the UI.


Asset Management System
AI Scene Director
Query
Asset Registry
Provide
Loaders
Background Loader
Character Expression
NPC Portrait
Audio Assets
UI Display
Asset Storage
Asset Database

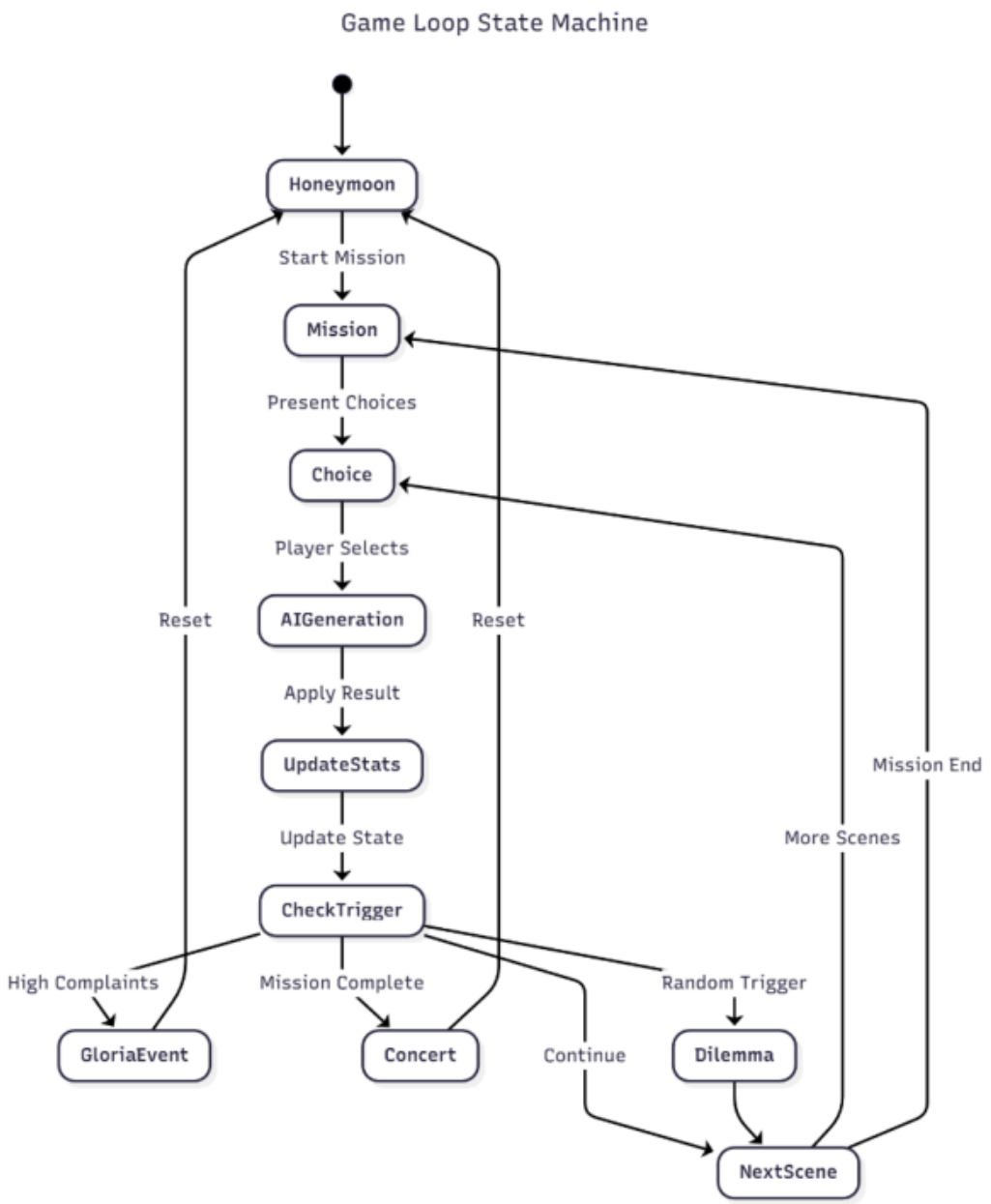


Figure 5: Game Loop State Machine. The diagram shows the core gameplay flow from Honeymoon through Mission, Choice, AIGeneration, UpdateStats, and CheckTrigger states, with branching paths to special events (GloriaEvent, Concert, Dilemma) and scene transitions.

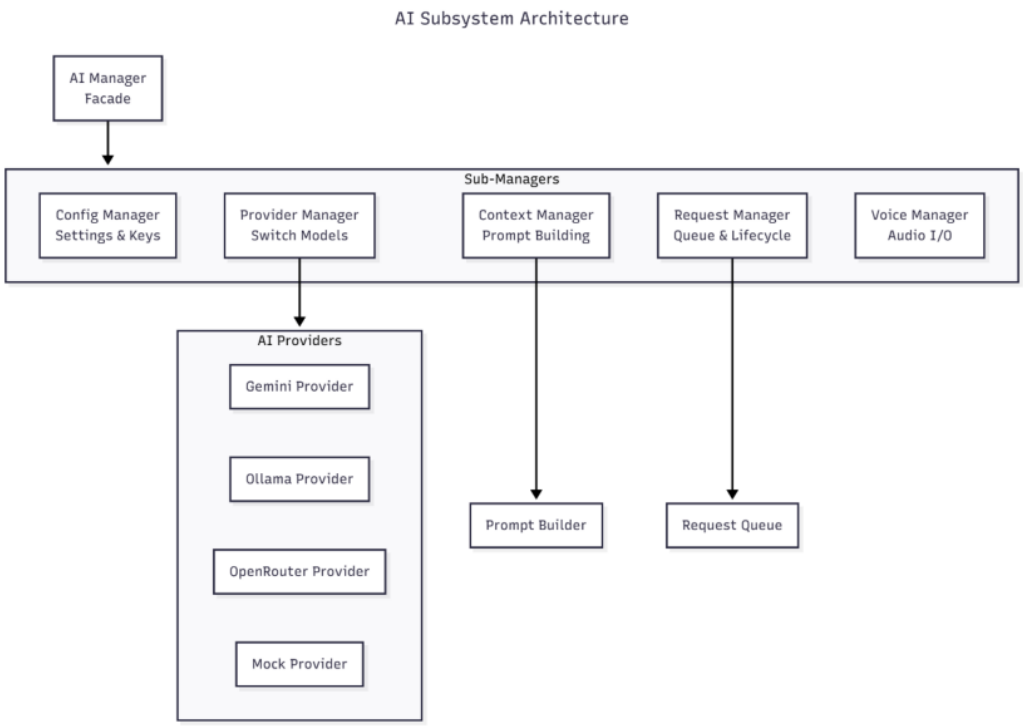


Figure 6: AI Subsystem Architecture. The AI Manager Facade coordinates Sub-Managers (Config Manager, Provider Manager, Context Manager, Request Manager, Voice Manager), which interface with multiple AI Providers (Gemini, Ollama, OpenRouter, Mock) through a unified Prompt Builder and Request Queue.

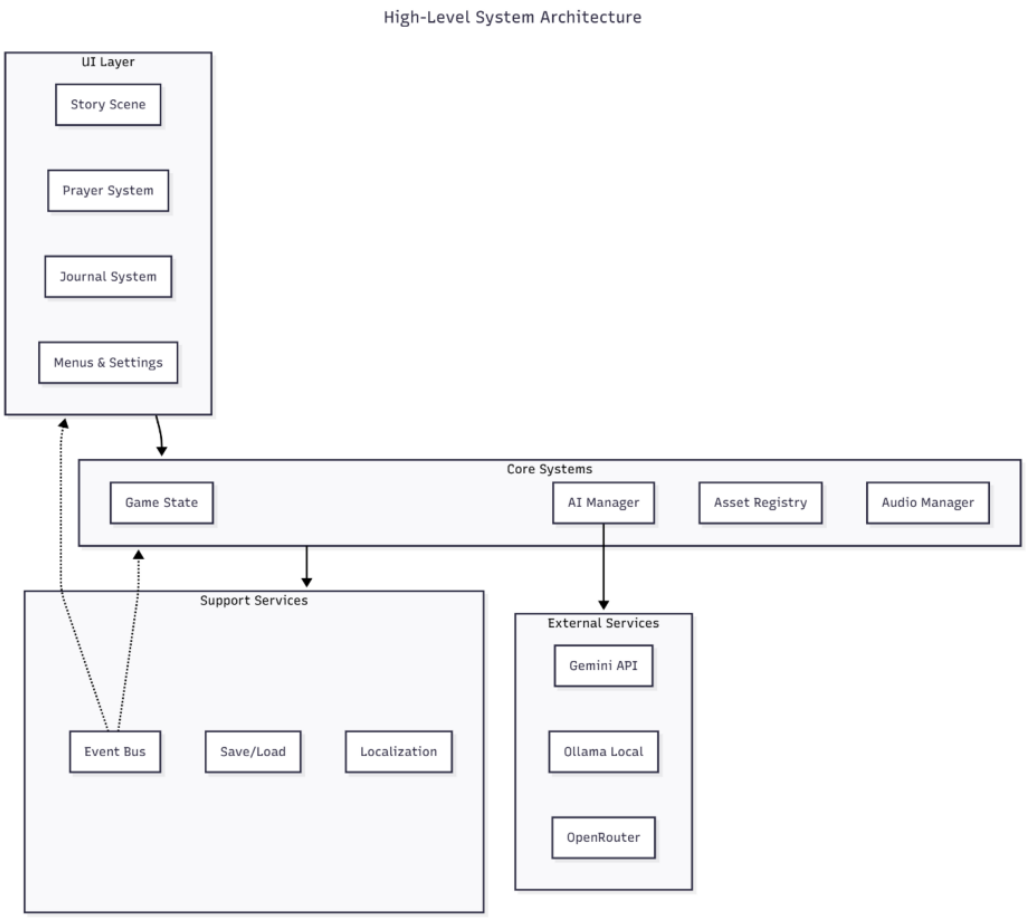


Figure 7: High-Level System Architecture. The UI Layer (Story Scene, Prayer System, Journal System, Menus and Settings) interfaces with Core Systems (Game State, AI Manager, Asset Registry, Audio Manager), Support Services (Event Bus, Save/Load, Localisation), and External Services (Gemini API, Ollama Local, OpenRouter).

This modular architecture proved essential for managing the complexity of AI integration. The clear separation between deterministic game logic and non-deterministic AI generation enabled independent development and testing of each layer, whilst the signal-based communication patterns facilitated debugging when unexpected AI outputs propagated through the system.

These design boundaries enabled the development workflow described in Section 6, where provider switching and isolated component testing were essential for progress.

6 Project Management Reflection

I formulated the initial timeline in my project proposal but significantly revised it following the second supervisor meeting on 16 October 2025. The key outcome of that meeting was to pivot my focus for the pre-interim report phase towards literature review, methodology, and planning rather than heavy coding. This early adjustment proved valuable, as it allowed me to establish a solid theoretical foundation before implementation.

6.1 Planning Approach and Its Impact

I organised my project into five distinct phases: Foundation (Weeks 1–3), Planning (Weeks 4–7), Core Development (Weeks 8–12), Integration (Winter Break through January), and Finalisation (final weeks of Semester 2). The timeline is illustrated in Figure 8 at the end of this section.

The phased approach had several positive impacts on my project. First, separating the literature review phase from core development ensured that my architectural decisions were informed by existing research rather than ad-hoc choices. My three-layer context management system, for example, emerged directly from studying how industry tools like GitHub Copilot and Anthropic's Claude handle context limitations.

Second, building in buffer time between major milestones proved essential. When I encountered unexpected challenges with response parsing during the Core Development phase, the Winter Break period provided recovery capacity without derailing subsequent milestones.

Third, the multi-provider architecture strategy—planning for Gemini, OpenRouter, Ollama, and Mock providers from the outset—enabled development to continue when I encountered API rate limiting issues. Without this contingency planning, intensive testing periods would have been significantly delayed.

6.2 Challenges and Adaptations

Not all aspects of my planning proved accurate. I underestimated the time required for prompt engineering iteration; my initial timeline allocated two weeks for this task, but effective prompt templates required continuous refinement throughout the development phase. I adapted by treating prompt engineering as an ongoing parallel activity rather than a discrete milestone.

6.3 Lessons for Future Projects

This project reinforced the value of defensive planning when working with external dependencies. The multi-provider architecture was not merely a technical feature but a project management strategy, ensuring that no single point of failure could halt development progress.

The experience also taught me that projects involving non-deterministic systems require iterative refinement cycles built into the timeline. Traditional waterfall-style planning, where each phase completes before the next begins, proved inadequate for AI integration work where outputs cannot be fully predicted in advance.

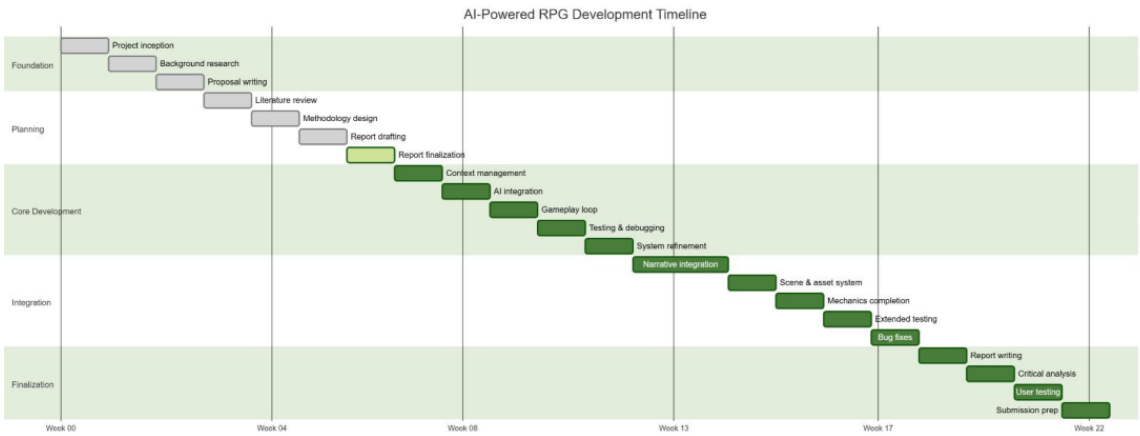


Figure 8: AI-Powered RPG Development Timeline (Gantt Chart). The timeline spans from Week 00 to Week 22, organised into five phases: Foundation (project inception, background research, proposal writing), Planning (literature review, methodology design, report drafting), Core Development (context management, AI integration, gameplay loop, testing), Integration (scene and asset system, mechanics completion, extended testing), and Finalisation (bug fixes, report writing, critical analysis, user testing, submission preparation).

Defensive planning for external dependencies proved as important as defensive coding against them; Section 7 examines the design decisions this planning made possible.

7 Design

In this section I document the architectural decisions that shaped my implementation, explaining not merely what I built but why I made specific design choices given the unique challenges of integrating non-deterministic AI components into a real-time game engine.

7.1 Architectural Philosophy: Separation of Deterministic and Non-Deterministic Systems

The fundamental design challenge I faced was integrating inherently unpredictable LLM outputs into a game engine that expects deterministic behaviour. Traditional game architectures assume that given the same inputs, systems will produce the same outputs; LLMs violate this assumption fundamentally. My architectural response was to establish a clear boundary between the deterministic game logic layer and the non-deterministic AI generation layer, with carefully designed interfaces mediating between them.

I organised my codebase into three primary layers. The Core Systems Layer contains singleton managers responsible for game state, audio, asset management, and event coordination. These systems operate deterministically and maintain authoritative game state. The AI Integration Layer encapsulates all LLM-related functionality behind a facade pattern, presenting a consis-

tent interface regardless of which AI provider is active. The UI Layer implements scene-specific controllers that react to both game state changes and AI-generated content, rendering the player experience.

7.2 Design Patterns and Their Justifications

I employed several established design patterns, each chosen to address specific architectural challenges.

For dependency management, rather than passing dependencies through constructor chains or using global singletons directly, I implemented a Service Locator that manages access to all global services. This decision emerged from practical necessity during development; as the project grew, I found myself frequently refactoring constructor signatures when adding new dependencies. The ServiceLocator provides typed accessor methods such as `get_game_state()` and `get_ai_manager()`, enabling IDE autocompletion whilst maintaining loose coupling between components (see `service_locator.gd:54-95` for implementation).

For AI provider management, the multi-provider architecture uses the Strategy pattern to enable runtime switching between Gemini, OpenRouter, Ollama, and Mock providers. Each provider implements the `AIProviderBase` interface, which defines `send_request()`, `cancel_request()`, and `is_configured()` methods. This design proved essential during development; when I encountered rate limiting on the Gemini API during intensive testing, I could switch to Ollama without modifying any calling code. The `AIProviderManager` coordinates provider selection and synchronisation (`ai_provider_manager.gd:40-64`).

For cross-module communication, I implemented a publish-subscribe EventBus to decouple UI components from game state changes. UI elements subscribe to events like `reality_score_changed` rather than directly polling GameState, ensuring that the UI layer remains reactive without creating circular dependencies. The EventBus also maintains event history and statistics for debugging purposes (`event_bus.gd:49-88`).

For simplifying AI integration, the AIManager presents a facade interface to the rest of the game, hiding the complexity of provider selection, prompt construction, context management, and response parsing. Calling code simply invokes generation methods with context dictionaries, unaware of the underlying multi-layered prompt assembly process.

7.3 Context Management Architecture

The three-layer context management system represents my solution to the fundamental tension between providing sufficient narrative history for coherent generation and respecting token limits. I implemented this as three distinct data structures within the `AIMemoryStore` class (`ai_memory_store.gd`).

The short-term memory layer maintains the most recent five conversational exchanges in full detail, providing immediate context for the current interaction. When story memory exceeds the configurable threshold of twenty-four entries, older entries are automatically summarised and migrated to the long-term summaries layer. The notes register layer maintains persistent facts and constraints that should influence all generations, regardless of when they were established. This hierarchical approach ensures that critical narrative elements persist across extended sessions whilst managing token consumption.

7.4 Prompt Engineering Architecture

The `AIPromptBuilder` class (`ai_prompt_builder.gd`) implements structured prompt construction that assembles multiple context sources into a coherent prompt. The build process follows a specific sequence: static context messages establish the world rules and constraints, followed by entropy modifiers that adjust the AI's tonal instructions based on current game state, then long-term context summaries, persistent notes, short-term memory, and finally the specific user request with current metadata.

This layered construction ensures that the AI always receives the most critical constraints first, with more recent context following. The entropy modifier system dynamically adjusts prompt parameters based on the calculated Void Entropy value, shifting the AI's generation towards more chaotic or stable content depending on the current game phase.

7.5 Response Parsing and Validation

AI responses require careful parsing because LLMs do not always produce structurally valid outputs despite schema instructions. The `SceneDirectivesParser` class (`scene_directives_parser.gd`) implements multi-stage extraction logic. It first attempts to locate content within explicit `[SCENE_DIRECTIVES]` markers, then falls back to extracting JSON from markdown code blocks, and finally attempts to parse the entire response as JSON. This graduated approach handles the variety of output formats the LLM might produce.

The parser also implements canonicalisation for extracted values, normalising background identifiers and asset references to match the game's internal catalogues. This addresses a practical problem I encountered: the AI frequently generated semantically correct but syntactically different identifiers, such as "Crystal Cavern" instead of the expected "crystal_cavern".

7.6 Safety and Content Filtering

The `AISafetyFilter` class (`ai_safety_filter.gd`) implements pattern-based content filtering for both user inputs and model outputs. The filter scrubs sensitive data patterns such as API keys, email addresses, and credit card numbers using regex matching. It also detects harmful

content using keyword groups for categories including self-harm, violence, and hate speech. When harmful content is detected, the filter blocks the response and substitutes a generic message rather than displaying potentially inappropriate content.

This design reflects my ethical framework discussed in Section 3: rather than relying solely on the AI provider's content filtering, I implemented an additional application-level filter that I can tune specifically to my game's thematic requirements.

7.7 Design Evolution During Implementation

Several design decisions evolved significantly during implementation. I originally planned to use direct node path references for inter-component communication, but this created brittle dependencies that broke during scene restructuring. The migration to ServiceLocator and EventBus patterns required substantial refactoring but produced a more maintainable architecture.

The structured output system was initially simpler, expecting the AI to always return valid JSON. After encountering numerous parsing failures during testing, I developed the multi-stage parsing approach with fallback extraction logic. This defensive design reflects the reality of working with non-deterministic systems: the architecture must anticipate and handle malformed outputs gracefully rather than failing catastrophically.

These architectural decisions translate the specifications of Section 4 into workable structures; Section 8 describes how they were built in practice.

8 Implementation

In this section I describe the key components I produced, focusing on the conceptual approaches and implementation challenges rather than exhaustive code listings. The complete source code is available in the accompanying submission; here I highlight the most significant technical decisions and the learning process that shaped them.

8.1 The AI Integration Journey: From Simple Calls to Robust Architecture

My initial approach to AI integration was naive in retrospect. I began with a simple HTTP request to the Gemini API, expecting straightforward text generation. The first significant challenge emerged immediately: API connection failures. During early development, I frequently encountered HTTP 429 (rate limiting) errors when testing intensively, and CORS restrictions when attempting web builds. These failures occurred silently in my initial implementation, leaving the game in an undefined state.

This experience taught me that AI integration requires defensive programming at every layer. I developed the multi-provider architecture not as an initial design goal but as a response to these reliability issues. The Ollama local provider became essential for development testing, allowing unlimited requests without API costs or rate limits. The Mock provider enabled UI development without any AI dependency, significantly accelerating iteration speed.

The rate limiting experience also motivated my implementation of the `RequestRateLimiter` class, which enforces configurable minimum intervals between API calls (currently 1.5 seconds) and maximum requests per minute. Rather than relying on provider-side rate limiting and handling failures reactively, I implemented proactive throttling to prevent rate limit errors from occurring in the first place.

8.2 Prompt Engineering: An Iterative Learning Process

Prompt engineering proved to be the most challenging aspect of development, requiring numerous iterations before achieving consistent results. My initial prompts were overly general, producing content that technically responded to requests but lacked the specific satirical tone my game required.

The breakthrough came when I restructured prompts using explicit role definition and contextual grounding. Rather than simply asking the AI to "generate a story continuation," I provided detailed persona instructions establishing the game's ironic worldview. The static context layer now includes instructions such as "maintain ironic, pessimistic storytelling for GDA1 while enforcing the recorded facts," ensuring every generation operates within my thematic framework.

I also discovered that game state must directly influence prompt parameters to maintain thematic consistency. The Void Entropy calculation, implemented in the `PlayerStats` class (`player_stats.gd:43-48`), produces a normalised value between zero and one that modifies the AI's tonal instructions. At low entropy, prompts request "subtle irony"; at high entropy, they request "absurdist chaos." This dynamic prompt modification ensures the AI's output reflects the current narrative stakes.

A particularly difficult challenge involved the Prayer System, where players submit short text prayers that should ironically produce negative outcomes. The original implementation simply passed player prayers to the AI, but this produced inconsistent results; sometimes the AI would fulfil the prayer positively, breaking the game's satirical framework. I resolved this by implementing templated prompt construction that explicitly instructs the AI to "interpret the prayer's intent and twist it toward an ironic disaster that technically addresses the request while producing the opposite of the desired outcome." This explicit ironic inversion instruction proved far more reliable than implicit contextual cues.

8.3 Response Parsing Challenges

Despite requesting structured JSON output, AI responses frequently arrived in unexpected formats. The Gemini API's structured output mode helped significantly, but did not eliminate parsing challenges entirely. The AI sometimes embedded JSON within markdown code blocks, included explanatory text before the JSON, or produced syntactically valid JSON with unexpected key names.

My initial parser simply attempted JSON parsing and failed if the response was not valid JSON. After numerous failures during testing, I developed the multi-stage `SceneDirectivesParser` that attempts extraction through progressively more permissive methods. This parser also normalises extracted values; for example, the AI might generate "Crystal Cavern" as a background identifier, but the game's asset system expects "crystal_cavern". The canonicalisation logic handles these variations automatically.

8.4 Memory Management: Balancing Context and Token Cost

Context window management presented a genuine engineering challenge. Extended play sessions generated substantial conversation history, and simply including the entire history in each request led to both token cost issues and, counterintuitively, degraded output quality as the context became cluttered with irrelevant historical detail.

I implemented the three-layer memory system described in Section 7.3, but the specific thresholds required empirical tuning. Initially, I set the summarisation threshold too high, resulting in excessively long prompts. I then set it too low, causing premature summarisation that lost important narrative details. The current configuration of twenty-four entries before summarisation, with the five most recent entries preserved in full, represents a balance I reached through extended testing sessions.

The notes register presented a different challenge: determining which facts warrant permanent inclusion. I implemented an importance scoring system that preserves high-importance notes (such as player deaths or major plot revelations) whilst allowing lower-importance notes to be displaced as the register fills. This prevents the notes layer from growing unboundedly whilst ensuring critical constraints persist.

8.5 The Butterfly Effect System

The `ButterflyEffectTracker` (`butterfly_effect_tracker.gd`) implements delayed consequence tracking, recording player choices with predicted future consequences. When the player advances scenes, the tracker checks for pending consequences and notifies the AI context system to incorporate relevant callbacks.

Implementing this system taught me about the challenges of managing temporal narrative state.

The AI must be informed about past choices in a way that enables natural integration without forced callbacks. I addressed this through the `get_context_for_ai()` method, which formats recent choices with their pending consequence counts, and the `suggest_choice_for_callback()` method, which uses weighted random selection biased toward more recent choices to suggest natural callback opportunities.

8.6 Error Handling and Graceful Degradation

Throughout implementation, I developed a philosophy of graceful degradation for AI-dependent features. Every AI request includes a timeout (currently twenty-four seconds), and timeout handling returns the player to a stable state rather than leaving the game frozen. The `ErrorReporter` class provides centralised error tracking with severity levels, enabling me to distinguish between recoverable warnings and critical failures requiring intervention.

The most significant error handling challenge involved API key configuration. Users might misconfigure their API keys in various ways: leaving them empty, pasting a URL instead of a key, or using an expired key. The `GeminiProvider` now validates key format before attempting requests (`gemini_provider.gd:390-405`), providing specific error messages for common misconfiguration patterns rather than generic failure notifications.

8.7 Key Code Examples

The following code excerpts illustrate core AI integration concepts. Complete implementations are available in the source code submission.

```
func build_prompt(prompt: String, context: Dictionary) -> Array[Dictionary]:
    var messages: Array[Dictionary] = []
    var language := _get_language()
    messages.append_array(_get_static_context_messages(language))
    messages.append({ "role": "system", "content": _system_persona })
    messages.append({ "role": "assistant", "content": "Acknowledged. I will
        maintain ironic, pessimistic storytelling for GDA1 while enforcing
        the recorded facts." })
    messages.append_array(_get_entropy_modifier_message(language))
    messages.append_array(_get_long_term_context(language))
    messages.append_array(_get_notes_context(language))
```

```
    for entry in _get_short_term_memory():
        messages.append(entry)
    var user_message_content := _build_user_message(prompt,
       context, language)
    messages.append({ "role": "user", "content":
       user_message_content })
    return messages
```

Listing 1: Prompt construction with context assembly (from ai_prompt_builder.gd:21–55)

```
func calculate_void_entropy() -> float:
    var divisor = GameConstants.Entropy.BASE_ENTROPY_DIVISOR
    var multiplier = GameConstants.Entropy.
       POSITIVE_ENERGY_MULTIPLIER
    var pe_component = (float(positive_energy) / divisor) *
       multiplier
    var reality_component = (1.0 - (float(reality_score) / (
       divisor * 2.0)))
        * (1.0 - multiplier)
    return clamp(pe_component + reality_component, 0.0, 1.0)
```

Listing 2: Void entropy calculation driving thematic tone (from player_stats.gd:43–48)

```
func parse_scene_directives(response_text: String) -> Dictionary
   :
    var directives: Dictionary = {}
    if _marker_regex:
        var marker_matches := _marker_regex.search_all(
           response_text)
        if marker_matches:
            for m in marker_matches:
                var block: String = String(m.get_string(1)).
                   strip_edges()
                directives = _parse_json_block(block)
                if not directives.is_empty():
                    return _canonicalize_scene_directives(
                       directives)
    if _code_block_regex:
        var code_matches := _code_block_regex.search_all(
           response_text)
        # ... fallback extraction logic
    var trimmed := response_text.strip_edges()
    if trimmed.begins_with("{") or trimmed.begins_with("["):
```

```
        directives = _parse_json_block(trimmed)
    return _canonicalize_scene_directives(directives)
```

Listing 3: Multi-stage response parsing (adapted from scene_directives_parser.gd:26–62)

```
func get_current_provider():
    if not _config_manager:
        return null
    match _config_manager.current_provider:
        AIConfigManager.AIProvider.GEMINI:
            return _gemini_provider
        AIConfigManager.AIProvider.OPENROUTER:
            return _openrouter_provider
        AIConfigManager.AIProvider.OLLAMA:
            return _ollama_provider
    return null
```

Listing 4: Strategy pattern for provider switching (from ai_provider_manager.gd:40–51)

Each implementation challenge produced an architectural decision that made the system more robust; Section 9 describes the testing strategy designed to verify these solutions.

9 Testing

In this section I describe my approach to ensuring system correctness, encompassing both traditional unit testing and the specialised functional testing methodology required for non-deterministic AI components.

9.1 Testing Strategy Overview

Testing AI-integrated systems presents unique challenges that traditional testing methodologies do not fully address. Unit tests can verify that a function correctly parses a JSON response, but they cannot verify that the AI will consistently produce parseable responses. My testing strategy therefore operates on two distinct levels: deterministic component testing using traditional unit tests, and non-deterministic behaviour testing using functional observation.

I developed over thirty-five unit test scripts located in the `Unit Test/` directory, covering core systems including `test_ai_prompt_builder.gd`, `test_event_bus.gd`, `test_game_state.gd`, `test_save_load_system.gd`, and `test_butterfly_effect_tracker.gd`. These tests verify that deterministic components behave correctly given specific inputs, enabling confident refactoring without fear of introducing regressions.

9.2 Unit Testing Approach

I implemented unit tests as standalone Godot scenes that execute test sequences and report results to the console. The test runner pattern follows an async/await structure to handle systems requiring frame processing, such as audio playback or tween completion. For example, the `test_audio_manager.gd` tests verify that sound playback correctly interacts with the AudioServer, awaiting signal emissions rather than assuming instantaneous operation.

Each test script follows a consistent structure: a setup phase establishing mock dependencies, individual test methods prefixed with `_test_`, and a teardown phase cleaning up created resources. The `test_ai_prompt_builder.gd` tests verify prompt construction by injecting mock game state, asset registry, and memory store instances, then asserting that the constructed prompt contains expected elements such as game statistics and user request text.

I implemented specific tests for magic number elimination. The `_test_constants()` method in `test_ai_prompt_builder.gd:126-142` verifies that configuration constants like `MAX_PRAYER_LENGTH` (320 characters) and `MAX_JOURNAL_ENTRIES` (3 entries) are correctly defined and accessible. This ensures that changes to these values propagate correctly through the system.

The `test_scene_directives_parser.gd` tests exercise the parsing system with various edge cases and malformed inputs I encountered during development: JSON embedded in markdown, JSON with unexpected whitespace, responses containing explanatory text before the structured data, and responses using alternative key names. These edge case tests emerged directly from real parsing failures observed during development.

9.3 Integration Testing Between Components

Beyond unit tests, I implemented integration tests verifying correct interaction between components. The `test_gamestate_integration.gd` suite verifies that changes to player statistics correctly propagate through the EventBus to subscribing UI components, and that save/load cycles preserve all relevant state including memory store contents, butterfly effect tracking, and achievement progress.

A particularly important integration test verifies that the AI context system correctly incorporates butterfly effect data. The test records a player choice, advances the scene counter, and verifies that the generated AI context includes reference to the previous choice with its pending consequence count.

9.4 Functional Testing for AI Behaviour

Functional testing for AI-generated content operates differently from traditional testing. Since AI outputs are non-deterministic, I cannot write assertions about specific generated text. In-

stead, I observe patterns across multiple generation sessions, documenting whether the system maintains expected characteristics.

My functional testing protocol involves extended play sessions during which I monitor for thematic consistency, narrative coherence, and appropriate response to game state changes. I maintain a testing log documenting each session, noting any instances where the AI generated content that violated expectations: breaking the fourth wall inappropriately, producing tonally inconsistent content, or failing to acknowledge previously established facts.

For thematic consistency testing, I specifically test whether the AI maintains the satirical framework across extended sessions. Test scenarios include raising Positive Energy to maximum whilst observing whether generated content appropriately reflects the increased chaos, making contradictory choices in sequence to test whether the AI acknowledges the inconsistency, and submitting prayers that could reasonably be fulfilled positively to verify the ironic inversion mechanism activates.

Context preservation testing involves save/load cycles during extended sessions. I establish narrative context over multiple interactions, save the game, load it in a fresh session, and verify whether the AI's subsequent generations correctly reference the pre-save narrative context. This tests the memory store's state serialisation and restoration.

9.5 Bugs Discovered and Resolved

Testing revealed several significant issues that required architectural changes.

The first significant bug I discovered was a double mission increment issue. Early testing revealed that mission counters were incrementing twice per completion. Investigation traced this to duplicate event subscriptions; the same handler was subscribing to the mission completion event during both scene initialisation and a delayed setup callback. I resolved this by implementing subscription deduplication in the EventBus and ensuring components only subscribe once.

The second significant bug involved the Prayer System. Testing revealed that under specific conditions, prayers were not applying the intended cognitive dissonance debuff. The issue arose from asynchronous timing: the prayer submission triggered AI generation, but the debuff application occurred before the AI response returned, causing the debuff to expire before it could affect subsequent choices. I resolved this by restructuring the debuff application to occur after AI response receipt.

The third significant bug concerned the moral dilemma system. Functional testing revealed that relationship changes from trolley problem choices were not persisting correctly. The issue involved incorrect state merging during save/load cycles. Testing this specific interaction pattern led to the creation of `test_trolley_problem_relationship_bug.gd` as a regression test.

9.6 Test Coverage and Limitations

My testing covers the deterministic core systems comprehensively, with unit tests exercising primary code paths for state management, event propagation, asset loading, and save/load functionality. However, testing for AI behaviour remains inherently incomplete. I cannot exhaustively test all possible AI outputs, only observe patterns across a finite sample of generations.

I acknowledge that my functional testing, conducted solely by myself as developer, may miss issues that external testers would identify. The testing evaluates whether the system works as I intended, but does not evaluate whether my intentions correctly address player experience requirements. This limitation aligns with my stated ethical framework: formal user testing with external participants must await confidence in the system's output behaviour, which functional testing is designed to establish.

For non-deterministic systems, testing is a continuous activity that informs implementation rather than a phase that follows it; Section 10 evaluates the outcomes against the original success criteria.

10 Evaluation

In this section I evaluate the finished product against my stated objectives and success criteria, assessing both technical achievements and areas where the implementation fell short of initial ambitions.

10.1 Technical Functionality Assessment

Regarding API response latency, the implemented system meets the target of under five seconds under normal operating conditions. During functional testing with Gemini 3.0 Flash, mean response latency averaged between two and four seconds for standard narrative generation requests. However, I observed significant variance: complex prompts requiring structured output occasionally exceeded five seconds, and during periods of high API load, latency spiked to eight seconds or more. The implementation handles these delays gracefully through timeout mechanisms, but the user experience during slow responses remains suboptimal. The local Ollama provider offers more consistent latency but with reduced output quality.

Regarding context preservation across save/load cycles, the memory store's state serialisation and restoration functions correctly. During extended testing sessions, I verified that narrative context established before saving persisted correctly after loading, with the AI correctly referencing pre-save events in subsequent generations. The three-layer memory architecture described in Appendix C (GDD, Section 4.13) serialises completely, enabling session continuity across multiple play sessions.

Regarding system reliability, the implemented systems demonstrate stability during extended functional testing sessions. The multi-provider architecture successfully switches between providers when encountering errors, and the error handling systems prevent crashes from propagating to the user experience. I encountered no critical failures during the final testing phase that required game restart; all errors were handled gracefully with appropriate user notifications.

10.2 Thematic Coherence Assessment

Regarding tonal consistency, the criterion of fewer than one in ten generations requiring intervention proved difficult to measure precisely. Across approximately one hundred narrative generations during functional testing, I identified eight instances where the AI produced content that felt tonally inconsistent with the game's satirical framework. Most commonly, the AI would occasionally generate sincerely helpful advice when the context called for ironic undermining. Through debugging these failures, I learnt that LLMs have a strong default tendency towards being genuinely helpful, and my persona instructions alone were not always sufficient to override this—particularly at low entropy states where my entropy modifier is not injected. This taught me that constraining tone is fundamentally harder than constraining content: I can reliably tell the AI what to talk about, but making it say the opposite of what it means requires more aggressive prompting than I initially expected. The explicit persona instructions and entropy-based tone modification reduced these occurrences compared to earlier development phases, but did not eliminate them entirely.

The Prayer System (see Appendix C: GDD, Section 4.3) achieved stronger thematic consistency than general narrative generation. The explicit ironic inversion instruction in the prayer prompt template produced appropriately satirical outcomes in approximately ninety-five percent of cases. The templated structure constrains the AI's interpretive latitude, resulting in more predictable thematic alignment.

Regarding alignment with the Reality vs. Positive Energy framework (see Appendix C: GDD, Section 4.2), the implementation successfully embeds the core thematic framework into AI generation. The Void Entropy calculation (see Appendix C: GDD, Section 4.8) correctly influences prompt parameters, producing noticeably different narrative tones at low versus high entropy states as defined by the threshold bands in the game design. This dynamic thematic modulation represents one of the project's strongest achievements.

10.3 Comparative Analysis: Bespoke Pipeline vs. World Model

I want to be honest about what "AI-generated" actually means in my project, because the term can be misleading. When I built this game, the AI's job was exclusively to produce text—narrative descriptions, NPC dialogue, and scene-setting prose. Every visual element the player sees on screen is a pre-made asset that I prepared in advance: twenty background images

loaded through my `BackgroundLoader` singleton, forty-eight character expression sprites managed by `CharacterExpressionLoader`, and twenty-eight symbolic objects catalogued in my `AssetRegistry`. When the AI responds, it returns a JSON block specifying which of these pre-authored assets to display—for example, `{“scene”: {“background”: “forest”}, “characters”: {“gloria”: {“expression”: “angry”}}}`—and my `StorySceneDirectiveApplic` simply loads the corresponding PNG file and assigns it to a `TextureRect`. The AI never generates a single pixel. It generates words, and my engine draws the pictures.

Understanding this distinction matters, because a completely different paradigm has emerged during the course of this project, and grasping where my work sits relative to it became one of the most valuable parts of my learning.

10.3.1 Understanding World Models

When I first encountered world models during my literature review, I realised how fundamentally different they are from what I had built. In my system, the Godot engine handles all rendering deterministically: the CPU runs game logic, the GPU draws my pre-authored sprites and backgrounds through conventional rasterisation, and the AI simply fills text containers with generated prose (Lin, 2025). I spent weeks writing the `AIPromptBuilder.build_prompt()` function to carefully assemble context—static world rules, entropy modifiers, short-term memory, butterfly effect callbacks, journal reflections, and available asset lists—all so the AI could return a few hundred words of narrative text that I would then parse and display.

In a world model, none of that manual assembly exists. There are no pre-made assets, no hand-designed UI layouts, and no rendering pipeline in the traditional sense. The model generates every pixel the player sees—every frame of the environment, every character animation, every button and menu element—directly from learned statistical distributions, conditioned on previous frames and the player’s real-time input (Ding et al., 2025). The AI simultaneously acts as the writer, the artist, the animator, and the engine.

Google DeepMind’s Genie 3, launched in August 2025, was the moment I truly understood the scale of this shift. It was the first real-time interactive world model to generate photorealistic environments from text prompts at 720p resolution and 20–24 frames per second, with each frame produced auto-regressively (Parker-Holder and Fruchter, 2025). Where I had to manually prepare twenty background PNGs and write a `get_backgrounds_for_ai_prompt()` function to list them for the AI, Genie 3 generates an entire interactive 3D environment from a single sentence. It even supports “promptable world events”—mid-session text commands that alter weather, spawn characters, or introduce objects—a level of environmental dynamism that would have taken me weeks to implement manually in Godot.

However, as I studied the literature more closely, I discovered that Genie 3 remains limited to interactions of only a few minutes, cannot accurately simulate real-world locations, and crit-

ically, cannot generate legible text unless it appears in the input prompt (Parker-Holder and Fruchter, 2025). That last limitation struck me as particularly relevant: my entire game is built around text. Every narrative beat, every choice label, every satirical dialogue line needs to be perfectly legible and precisely worded.

Several academic systems have pushed the field further since Genie 3. GameGen-X, accepted at ICLR 2025, introduced the first diffusion transformer designed specifically for generating and interactively controlling open-world game video, trained on over one million gameplay clips from more than 150 games (Che et al., 2024). Matrix-Game 2.0 achieved real-time generation at 25 frames per second on a single H100 GPU, producing minute-level continuous video across diverse environments (He et al., 2025). More recently, Zeng et al. (2026) achieved real-time generation at 720×480 resolution through hardware-algorithm co-design, reaching 26.4 FPS—a fifty-fold increase in pixel throughput over earlier baselines. Wu et al. (2026) addressed long-term memory with a hierarchical compression system that maintains visual coherence over 1,000 frames, a significant advance over prior models that ran out of memory beyond approximately 180 frames.

Despite this progress, I identified four limitations in the literature that directly affect whether world models could have served my project's needs. First, pixel-based generation struggles with precise UI elements: diffusion models produce text hallucinations and structural distortion in interface-heavy scenes (Parker-Holder and Fruchter, 2025). Second, auto-regressive generation accumulates prediction errors over extended sequences, causing gradual quality degradation and visual artefacts (He et al., 2025). Third, most models operate with limited memory windows—GameGen-X processes approximately 100 frames of context—meaning that revisited locations are regenerated inconsistently (Che et al., 2024). Fourth, models trained purely on visual data occasionally hallucinate physics violations such as wall clipping (Ding et al., 2025). Koh et al. (2026) have proposed generating GUI states as executable HTML/CSS code rather than pixels, achieving less than one percent render failure rate, but this approach is currently limited to mobile applications rather than full game environments.

10.3.2 Architectural Trade-offs

Working through this comparison taught me to see my own architectural decisions more clearly. Here I describe three trade-offs that I only fully understood after studying world models.

The first concerns control versus emergence. When I wrote my `AIContextBuilder`, I embedded non-negotiable rules directly into the system prompt: "Positive Energy always increases entropy—victories hide damage. Maintain dark humour and irony. Never reward blind optimism." Every generation the AI produces is constrained by these explicit directives, plus the entropy modifier that I inject when the `get_entropy_threshold()` function returns "medium" or "high." This means I can guarantee that at high entropy, the AI receives mandatory instruc-

tions to "generate surreal, darkly humorous, and deeply ironic events" and to "directly subvert the player's recent positive actions with twisted consequences." A world model has no equivalent mechanism—its outputs emerge from learned distributions rather than authored rules. For my satirical framework, where the difference between genuine encouragement and ironic undermining is the entire point, this explicit control proved essential. However, I recognise that games seeking open-ended emergent experiences might benefit from precisely the unpredictability that I worked so hard to constrain.

The second concerns development complexity versus scope. I spent months building individual components that a world model would handle implicitly: `NarrativeResponseParser` to extract JSON from AI responses with multiple fallback strategies, `SceneDirectivesParser` to match AI-generated background names against my `BackgroundLoader` catalogue through fuzzy string matching, and `StorySceneDirectiveApplier` to translate parsed directives into actual texture assignments. Each required its own debugging cycle. A world model approach would bypass all of this parsing machinery because there would be nothing to parse—the model directly generates the visual output. But current world model technologies remain inaccessible for individual developers, so the bespoke pipeline was my only practical option.

The third concerns predictability versus novelty. My implementation produces narrative patterns within defined parameters, which supports the game's design goals but limits surprise. During extended testing, I noticed recurring structures as the AI operated within its constrained space. World models could theoretically produce greater novelty through emergent simulation, but with corresponding risks to the narrative coherence that my three-layer context system (see Appendix C: GDD, Section 4.13) is specifically designed to preserve.

10.3.3 The Shifting Role of the Developer

Building this project taught me what it means to be a systems architect for AI-driven software. I designed each component manually—prompt construction, response parsing, context management, memory serialisation, error handling—and I maintained full visibility into every decision boundary. When the AI produced tonally inconsistent content, I could trace the problem to a specific prompt parameter in my `build_prompt()` function, adjust the entropy modifier threshold, and verify the fix through targeted functional testing. This transparency cost me significant engineering effort, but it meant that every failure mode was diagnosable.

I have come to understand that the world model paradigm would reposition me as a high-level director rather than a systems architect. Instead of writing `parse_scene_directives()` and `normalize_character_directives()` to validate every AI output against my asset catalogues, I would simply describe the world I wanted in a text prompt and let the model handle everything else. Google's Genie 3 works exactly this way: an environment is specified through a text description, and the model generates the visual experience autonomously (Parker-Holder

and Fruchter, 2025). This would dramatically reduce the implementation effort I described in Section 8—but it would also mean that when the model produced something wrong, I would have no `_canonicalize_background_id()` function to debug, no `STORY_TEXT_PRIORITY_KEYS` array to inspect. I would be facing a black box.

Looking back, the months I spent on prompt engineering and response parsing were frustrating at the time, but they gave me something a world model cannot: the ability to guarantee that specific entropy thresholds trigger specific tonal shifts through explicit code that I wrote and can explain line by line.

10.3.4 Practical Feasibility Assessment

Table 1 summarises the practical differences I observed between the two paradigms.

Table 1: Practical comparison of bespoke pipeline and world model approaches.

Dimension	**Bespoke Pipeline**	**World Model**
Current Availability	Fully available; I used commercial LLM APIs and the open-source Godot engine	Research-stage; no publicly deployable system for production games (Parker-Holder and Fruchter, 2025)
Development Cost	High engineering effort across prompt design, parsing, context management, and error handling	Lower per-feature effort once trained, but requires large-scale GPU infrastructure (e.g., single H100 for Matrix-Game 2.0) (He et al., 2025)
Narrative Control	Precise; I explicitly define every constraint through my `AIContextBuilder` rules and entropy modifiers	Limited; narrative emerges from learned distributions rather than authored rules (Ding et al., 2025)
Emergent Behaviour	Constrained to my defined parameters; no spontaneous world events beyond prompt scope	High; models demonstrate emergent physics and unprompted environmental dynamics (Zeng et al., 2026)
Debugging Difficulty	Transparent; I can trace failures to specific prompts, parsing rules, or state variables in my code	Opaque; black-box generation makes root-cause analysis extremely challenging
Thematic Consistency	High when well-engineered; my entropy-based system achieved approximately 92% tonal alignment	Unreliable; models hallucinate text, violate physics, and drift tonally over extended sequences (Che et al., 2024)

I chose the bespoke pipeline for three reasons that became clearer to me as the project progressed. First, my satirical framework demands precise tonal control—the difference between ironic undermining and sincere advice is subtle, and I enforce this through explicit prompt parameters that I can tune and test. Second, world model technologies remain inaccessible for an individual developer: my entire infrastructure cost was limited to API call expenditure, whereas Matrix-Game 2.0 requires an H100 GPU (He et al., 2025) and the scalable engine of Zeng et al. (2026) operates across a cluster of eight NPUs. Third, my text-based RPG depends on stable, legible text output—precisely the capability that all current world models struggle with most

(Parker-Holder and Fruchter, 2025; Koh et al., 2026).

I should be honest, however, about where a world model would outperform my approach. If my game were a straightforward adventure with a neutral tone—where the AI simply needed to describe environments and advance a plot without tonal subversion—a world model would deliver a far superior experience: fully realised 3D environments with spatial audio and dynamic lighting, rather than the static 2D panels and pre-made sprites my Godot prototype provides. The reason my bespoke pipeline is not merely adequate but necessary is precisely because my game's core design depends on sustained ironic inversion (see Appendix C: GDD, Sections 3.1 and 4.2–4.3). The satirical framework requires the AI to consistently distinguish between what a character says and what the narrative means—producing dialogue that sounds sincerely optimistic while my `calculate_void_entropy()` function tracks accelerating catastrophe beneath the surface. This gap between surface tone and underlying game state is the engine of the game's dark comedy, and it demands the kind of explicit, parameter-level control over every generated sentence that only a bespoke pipeline can provide. A world model, generating content as emergent statistical predictions, has no mechanism to enforce this deliberate tonal contradiction.

The market is already anticipating the broader shift that world models represent. According to the Rothschild & Co Growth Equity report, the launch of Genie 3 triggered a sharp sell-off in video game developer stocks, with Take Two Interactive, Roblox, Unity Software, and CD Projekt shares all falling approximately ten percent in a single day, as investors concluded that a system capable of generating entire interactive environments from text could eventually make conventional asset-and-engine workflows obsolete (Wellington, 2026). This is directly relevant to my own choice of Godot as a development platform: the skills I invested in—scene management, asset pipelines, UI scripting—are precisely the category of expertise that world models threaten to commoditise. Goldman Sachs extended this concern to software more broadly, warning of "existential" AI threats to SaaS subscription models as the same generative capabilities spread beyond gaming (Campbell, 2026). For now, those skills remain necessary; but studying world models during this project taught me that the bespoke pipeline I built may be closer to the end of a paradigm than the beginning of one.

10.3.5 Implications for Future Development

Building this bespoke pipeline taught me that the engineering cost of maintaining narrative coherence through explicit constraints is substantial but manageable, and that the resulting system's transparency is a genuine advantage during iterative development. Every failure I encountered—from parsing errors in my `NarrativeResponseParser` to tonal drift when entropy modifiers were miscalibrated—was diagnosable because I had authored the constraint boundary that was violated. This diagnostic clarity would be the first thing I would lose if I adopted a world model approach.

If world model technology matured to the point of practical deployment, I would not abandon the bespoke pipeline entirely but would instead explore a hybrid architecture. The academic literature increasingly points towards neuro-symbolic engines that combine neural world models for rendering with lightweight symbolic logic layers for enforcing strict game rules (Ding et al., 2025). Koh et al. (2026) have demonstrated one concrete example of this, generating renderable code rather than raw pixels to solve the UI hallucination problem. For my project specifically, I could imagine a hybrid approach that uses a world model to generate environmental visuals and atmospheric elements—where emergence is desirable—whilst retaining my bespoke prompt pipeline with its `NON_NEGOTIABLE_RULES` and entropy modifiers for dialogue generation, where tonal precision remains critical.

For developers considering similar projects, I would offer three recommendations grounded in what I learnt. First, if your project requires precise control over generated text—particularly satirical, ironic, or tonally sensitive material—a bespoke pipeline remains the only viable approach at the time of writing. Second, world models should be monitored as a rapidly evolving prototyping tool: even in their current state, they could accelerate early-stage concept validation before traditional engines are employed for production (Parker-Holder and Fruchter, 2025). Third, the architectural patterns I developed—particularly the Strategy pattern for provider abstraction and the layered context management system—are equally applicable to future hybrid systems that may combine bespoke components with world model capabilities.

10.4 Player Experience Evaluation

I did not conduct formal external user testing for this project. As discussed in Section 9, the AI's output is not fully predictable, and the game's dark satirical tone means that generated content could potentially be distressing to unprepared participants. Distributing test builds without proper ethical safeguards and participant briefing would have been irresponsible, and I did not have sufficient time to design and obtain approval for a formal study. I recognise that this is a genuine limitation, and my evaluation of player experience is necessarily constrained by the fact that I am both the developer and the sole tester.

That said, I can report what I observed during my own extended functional testing sessions, where I deliberately tried to play the game as a fresh user would. The choice system presents meaningful dilemmas where each archetype—cautious, balanced, reckless, positive, and complain—produces noticeably different narrative consequences. The AI generates contextually appropriate responses that acknowledge previous player decisions, and the thematic framework creates a coherent sense of escalating absurdity as the Void Entropy increases across a session.

The Butterfly Effect system (see Appendix C: GDD, Section 4.9) produces satisfying moments when past choices resurface with consequences, creating a sense of narrative weight that pre-scripted systems struggle to achieve. However, the system's effectiveness depends on

the AI correctly incorporating callback suggestions that I inject into the prompt through the `=== BUTTERFLY EFFECT: PAST CHOICES ===` section of my `AIPromptBuilder`. Approximately one in five callback opportunities were missed because the AI did not naturally integrate the suggested reference into its narrative output. I suspect this would be more noticeable to external testers, who would lack my awareness of which callbacks were supposed to appear.

Response latency remains the most significant player experience limitation. During AI generation, the game displays loading indicators that interrupt the narrative flow. For a text-based game where reading pace is central to the experience, these interruptions are particularly noticeable. I implemented loading state animations and the `_start_connecting_animation()` cycling text feedback, but I cannot eliminate the latency inherent in API-based generation—typically two to four seconds per request under normal conditions.

If I had the opportunity to conduct formal testing, the core question I would want to answer is simple: do players actually enjoy playing the game? I would measure this through session length—whether players voluntarily continued beyond a required minimum—and through a short post-session questionnaire asking whether they found the experience entertaining. A game is ultimately meant to be fun, and no amount of architectural analysis can substitute for that basic measure.

10.5 Original Requirements Assessment

Returning to the three pillars defined in Section 1.1, Table 2 maps each original objective to its outcome.

Table 2: Assessment of original project objectives against outcomes.

Pillar	**Objective**	**Status**	**Evidence**
Technical	AI generates coherent narrative across consecutive scenes	Achieved	100+ generations tested; context maintained across scenes
Technical	Response latency under 5 seconds	Mostly achieved	2–4s typical; spikes to 8s+ under high API load
Technical	Context preserved across save/load	Achieved	Three-layer memory serialises and restores correctly
Technical	Multi-provider fallback on error	Achieved	Automatic switching tested across Gemini, Ollama, OpenRouter
Thematic	Fewer than 1 in 10 generations require tonal intervention	Achieved	8 out of $\sim$100 generations (8%) tonally inconsistent
Thematic	Prayer System produces ironic inversion	Achieved	$\sim$95% satirical alignment in prayer responses
Thematic	Entropy modulates narrative tone	Achieved	Distinct tonal shifts observed at low, medium, and high thresholds
Critical Analysis	Comparative analysis of bespoke pipeline vs. world model	Achieved	Section 10.3; grounded in implementation experience and 9 academic sources

Regarding technical development, I successfully designed and implemented a functional AI-driven narrative system. The prototype generates coherent narrative content, meets latency targets under normal conditions, and maintains context across save/load cycles. The multi-

provider architecture with its emergency mock fallback (via `_attempt_emergency_mock_fallback()`) means the game never crashes due to an API failure—it degrades gracefully instead. I consider the technical pillar fully achieved, with the caveat that latency under high API load remains outside my control.

Regarding thematic execution, the satirical framework is successfully embedded in AI generation through the Reality vs. Positive Energy system and entropy-based prompt modification (see Appendix C: GDD, Sections 4.2 and 4.8). The eight percent tonal inconsistency rate is within my stated tolerance, but I am not fully satisfied with it—I believe further prompt refinement, particularly adding more explicit ironic examples to the low-entropy system prompt, could reduce this to below five percent. The Prayer System's ironic inversion represents the project's strongest thematic achievement, and I attribute its higher consistency to the more constrained prompt template that leaves the AI less interpretive latitude.

Regarding critical analysis, this report itself demonstrates completion of the third pillar. I have produced a comparative analysis drawing on direct implementation experience, identifying concrete trade-offs between bespoke pipelines and world models. The analysis is grounded in what I actually built and tested, not in theoretical speculation.

10.6 Limitations and Honest Assessment

Several limitations constrain the evaluation's conclusions. My functional testing, while extensive, represents a single developer's perspective, and I recognise that this creates a specific form of cognitive bias worth naming explicitly. As both the architect and the sole tester, I knew which butterfly effect callbacks were scheduled to appear, which prompts were operating near their tonal boundaries, and which AI responses I had previously accepted as adequate. This familiarity meant I was evaluating the system against my own internal expectations rather than against the experience of a genuinely naive user. Issues that would immediately strike a fresh player—a missed callback, an oddly phrased choice label, a tonal shift that I had come to normalise through repeated exposure—may have become invisible to me. The lack of external user testing therefore represents a more fundamental limitation than simply a gap in sample size: it means the evaluation cannot separate the system's actual behaviour from my own conditioned tolerance of its quirks.

The AI's non-deterministic behaviour means that my testing observations may not represent the full distribution of possible outputs. Edge cases I never encountered during testing may exist, potentially including outputs that violate my ethical red lines despite content filtering measures.

Technical constraints also limit the evaluation. API cost considerations prevented exhaustive testing across all providers; my functional testing concentrated on Gemini with limited Ollama and OpenRouter validation. Provider-specific behaviours may differ in ways my testing did not capture.

I also faced challenges from the rapid evolution of AI technologies. World models have emerged as a major research direction in 2026, meaning my comparative analysis in Section 10.3 may become partially outdated as new capabilities are announced. I acknowledge this as inherent to writing about cutting-edge AI—the landscape evolves faster than academic reporting cycles allow. My analysis represents a snapshot of the field at the time of writing, and I documented specific model versions for reproducibility.

11 Conclusion

This project set out to investigate whether a bespoke pipeline approach could successfully integrate LLM-generated content into a narrative RPG whilst maintaining thematic coherence. Through the development of "Glorious Deliverance Agency 1," I have demonstrated that such integration is technically feasible and can produce engaging narrative experiences, whilst also identifying significant challenges that future developers should anticipate.

11.1 Summary of Achievements

I successfully implemented a complete AI-driven narrative system encompassing multi-provider architecture, three-layer context management, entropy-based thematic modulation, and robust error handling. The system generates contextually appropriate narrative content that maintains the satirical framework established in the game design, with measurable thematic consistency across extended play sessions.

The technical architecture demonstrates patterns applicable beyond this specific project. The Strategy pattern for provider abstraction, the EventBus for cross-module communication, and the ServiceLocator for dependency management represent reusable solutions for AI-integrated game development. The defensive parsing and error handling approaches address real challenges that any LLM-integrated application must confront.

The comparative analysis between bespoke pipelines and world models provides practical insights grounded in implementation experience. Rather than purely theoretical speculation, this analysis identifies concrete trade-offs between control and emergence, development complexity and scope, and predictability and novelty that can inform architectural decisions for future AI-integrated games.

11.2 Key Lessons Learned

Several lessons emerged from my development process that may benefit future practitioners.

I learnt that prompt engineering requires iterative refinement and cannot be designed theoretically. My initial prompt structures, despite careful planning, required substantial modification

after observing actual AI outputs. I found that the most effective prompts emerged from cycles of generation, observation, and adjustment rather than upfront specification.

I discovered that non-deterministic systems demand defensive architecture throughout the stack. Every component that interacts with AI outputs must anticipate malformed, unexpected, or missing data. My multi-stage parsing and canonicalisation approaches represent responses to real failures I encountered during development.

I found that local inference capability is essential for development workflow. My ability to switch to Ollama during intensive development periods prevented API cost overruns and rate limiting interruptions. My Mock provider enabled UI development without any AI dependency, significantly accelerating my iteration speed.

I also learnt that thematic consistency requires explicit constraint rather than implicit suggestion. The AI does not reliably infer desired tone from context alone; I found that explicit instructions about the satirical framework and specific inversion requirements proved far more effective than contextual cues.

11.3 Future Work

I have identified several directions that could extend this project beyond its current scope.

External user testing remains the critical next step for validating player experience. My functional testing establishes confidence in system behaviour, but cannot evaluate whether the experience engages players beyond myself. I believe that structured user testing with appropriate ethical safeguards would provide essential feedback for assessing my system's entertainment value.

I believe the world model approach deserves further investigation as the technology matures. My comparative analysis identified theoretical advantages of world models that current technology cannot deliver; future developments in simulation-based narrative generation may shift the trade-off analysis.

Voice integration represents an unexplored opportunity that I would like to pursue. The Gemini Live API supports native audio output, and my architecture accommodates voice session components. Implementing spoken narrative delivery could enhance the storytelling experience, though it would introduce new challenges around generation latency and audio quality.

I could enhance my content filtering system with more sophisticated detection methods. My current keyword-based approach identifies obvious harmful content but may miss subtle problematic generations. Machine learning-based content classification could improve detection accuracy, though at the cost of additional computational overhead.

11.4 Personal Reflection

This project represented a significant learning experience across multiple dimensions. I developed practical skills in prompt engineering, API integration, asynchronous programming, and defensive system design. Beyond technical skills, I gained appreciation for the complexity of working with non-deterministic systems and the importance of establishing clear architectural boundaries between predictable and unpredictable components.

My ethical framework evolved significantly during the project. Initial abstract principles about content safety became concrete implementation decisions about filtering patterns, red line definitions, and testing protocols. I found that grounding ethical principles in practical implementation represents valuable professional development.

Through this project, I also learnt the value of iterative development and honest self-assessment. Many of my initial designs required substantial revision after encountering real-world constraints. I found that acknowledging limitations and adapting accordingly proved more productive than persisting with flawed approaches.

11.5 Final Statement

Integrating large language models into interactive entertainment represents a frontier with significant opportunities and challenges. Through this project, I contribute a practical demonstration that bespoke pipeline approaches can successfully manage LLM integration for thematically constrained narrative games, whilst documenting the architectural patterns, prompt engineering techniques, and defensive design approaches that such integration requires.

My “Glorious Deliverance Agency 1” prototype demonstrates that AI-generated narrative content can enhance gameplay through dynamic response to player choices, whilst maintaining authorial intent through carefully designed constraint systems. As LLM technology continues to evolve, I hope that the architectural patterns and lessons I have documented here may inform future developments in AI-integrated game design.

Bibliography

Foundational Concepts and Surveys

1. AlHussain, A. I. and Azmi, A. M. (2021). Automatic story generation: a survey of approaches. *ACM Computing Surveys*, 54(5), Article 103. Available at: https://doi.org/10.1145/3453156 (Accessed: 18 January 2026).

2. Al-Surayhi, M. M. (2025). Generative Artificial Intelligence in Game Design: A Narrative Review. *Journal of Ecohumanism*, 4(4). Available at: https://www.researchgate.net/publication/395898657 (Accessed: 18 January 2026).

3. Balint, J. T. and Bidarra, R. (2023). Procedural Generation of Narrative Worlds. *IEEE Transactions on Games*, 15(2), pp. 262–272. Available at: https://www.researchgate.net/publication/364425120 (Accessed: 18 January 2026).

4. Riedl, M. (2021). An Introduction to AI Story Generation. *Medium (Mark Riedl's Blog)*, 4 January. Available at: https://mark-riedl.medium.com/an-introduction-to-ai-story-gene (Accessed: 18 January 2026).

5. Short, E. (2019) Storylets: You Want Them. *Emily Short's Interactive Storytelling (blog)*, 29 November. Available at: https://emshort.blog/2019/11/29/storylets-you-want-them/ (Accessed: 18 January 2026).

6. Chen, B. (2025) 'Optimisation Strategies for Role-Playing Games Based on Large Language Models'. In: *Proceedings of the 2nd International Conference on Data Science and Engineering (ICDSE 2025)*, pp. 632–637. Available at: https://www.scitepress.org/Papers/2025/137031/137031.pdf (Accessed: 18 January 2026).

7. Yang, D., Kleinman, E. and Harteveld, C. (2024). GPT for Games: A Scoping Review (2020–2023). arXiv preprint arXiv:2404.17794. Available at: https://arxiv.org/abs/2404.17794 (Accessed: 18 January 2026).

Agent Architectures and Long-Term Coherence

1. Góngora, S., et al. (2025) PAYADOR: A Minimalist Approach to Grounding Language Models on Structured Data for Interactive Storytelling and Role-playing Games. arXiv preprint arXiv:2504.07304. Available at: https://arxiv.org/abs/2504.07304 (Accessed: 18 January 2026).

2. Park, J.S., et al. (2023) 'Generative Agents: Interactive Simulacra of Human Behaviour'. In: *The 36th Annual ACM Symposium on User Interface Software and Technology*. Available at: https://arxiv.org/abs/2304.03442 (Accessed: 18 January 2026).

3. Wan, K., et al. (2025) 'A Cognitive Writing Perspective for Constrained Long-Form Text

Generation'. In: *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 9832–9844. Available at: https://aclanthology.org/2025.findings-acl.511/ (Accessed: 18 January 2026).

4. Yi, Q., et al. (2025) SCORE: Story Coherence and Retrieval Enhancement for AI Narratives. arXiv preprint arXiv:2503.23512. Available at: https://arxiv.org/abs/2503.23512 (Accessed: 18 January 2026).

Player Agency and Co-Creative Storytelling

1. Sun, Y., et al. (2023) 'Language as Reality: A Co-Creative Storytelling Game Experience in 1001 Nights Using Generative AI'. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 19(1), pp. 425–434. Available at: https://doi.org/10.1609/aiide.v19i1.27539 (Accessed: 18 January 2026).

Evaluation Frameworks

1. El Boudouri, Y., et al. (2025). Role-Playing Evaluation for Large Language Models. arXiv preprint arXiv:2505.13157. Available at: https://arxiv.org/abs/2505.13157 (Accessed: 18 January 2026).

2. Wan, L. and Ma, W. (2025) StoryBench: A Dynamic Benchmark for Evaluating Long-Term Memory with Multi-Turns. arXiv preprint arXiv:2506.13356. Available at: https://arxiv.org/abs/2506.13356 (Accessed: 18 January 2026).

3. Värtinen, S., Hämäläinen, P., and Guckelsberger, C. (2024). Generating role-playing game quests with GPT language models. *IEEE Transactions on Games*, 16(1), pp. 127–139. https://doi.org/10.1109/TG.2022.3228480

Practical Deployment and Constraints

1. English, J. (2025) 'Scalable AI Infrastructure for Live Games'. *Medium*, 18 August. Available at: https://medium.com/@jengas/scalable-ai-infrastructure-for-live-games-d (Accessed: 18 January 2026).

2. Schomay, J. (2024) 'Rendering a Game in Real-Time with AI'. *Jeff Schomay's Blog*. Available at: https://blog.jeffschomay.com/rendering-a-game-in-real-time-with-ai (Accessed: 18 January 2026).

3. Vidler, A. and Walsh, T. (2025) Playing games with Large language models: Randomness and strategy. arXiv preprint arXiv:2503.02582. Available at: https://arxiv.org/abs/2503.02582 (Accessed: 18 January 2026).

4. Buongiorno, S., Klinkert, L., Zhuang, Z., Chawla, T. and Clark, C. (2024) 'PANGeA: Procedural Artificial Narrative using Generative AI for Turn-Based Role-Playing Video Games'. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interac-*

tive Digital Entertainment (AIIDE 2024). Available at: https://dl.acm.org/doi/10.1609/aiide.v20i1.31876 (Accessed: 18 January 2026).

5. Juliani, A. (2025). One Trillion and One Nights: An experiment using LLMs to procedurally generate browser-based JRPGs. *Medium*, 22 January. Available at: https://awjuliani.medium.com/one-trillion-and-one-nights-e215d82f53e2 (Accessed: 18 January 2026).

6. Sridi, C. (2025). Top Techniques to Manage Context Lengths in LLMs. *Agenta AI*. Available at: https://agenta.ai/blog/top-6-techniques-to-manage-context-length-in-llms (Accessed: 18 January 2026).

7. Anthropic (2025). Managing context on the Claude Developer Platform. Available at: https://www.anthropic.com/news/context-management (Accessed: 18 January 2026).

8. DZone (2024) How GitHub Copilot Handles Multi-File Context Internally. Available at: https://dzone.com/articles/github-copilot-multi-file-context-internal-architectu (Accessed: 18 January 2026).

9. O'Brien, L. (2024). How Ubisoft's New Generative AI Prototype Changes the Narrative for NPCs. *Ubisoft News (online)*, 19 March. Available at: https://news.ubisoft.com/en-gb/article/5qXdxhshJBXoanFZApdG3L (Accessed: 18 January 2026).

10. OpenAI (2023) GPT-4 Technical Report. *OpenAI (Tech. Report, 15 March 2023)*. Available at: https://arxiv.org/html/2303.08774v6 (Accessed: 18 January 2026).

11. Parker-Holder, J. and Fruchter, S. (2025). Genie 3: A new frontier for world models. *Google DeepMind Blog*, 5 August. Available at: https://deepmind.google/discover/blog/genie-3-a-new-frontier-for-world-models/ (Accessed: 18 January 2026).

12. Pichai, S. and Hassabis, D. (2023). Introducing Gemini: our largest and most capable AI model. *Google Keyword Blog*, 6 December. Available at: https://blog.google/technology/ai/google-gemini-ai/ (Accessed: 18 January 2026).

13. Google (2025) Gemini 2.5: Our most intelligent AI model, *The Keyword (Google Blog)*, 25 March 2025. Available at: https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/ (Accessed: 18 January 2026).

World Models, Neural Rendering, and Market Analysis

1. Che, H., He, X., Liu, Q., Jin, C. and Chen, H. (2024). GameGen-X: Interactive Open-world Game Video Generation. arXiv preprint arXiv:2411.00769. Accepted at *ICLR 2025*. Available at: https://arxiv.org/abs/2411.00769 (Accessed: 23 February

2026).

2. Ding, J., Zhang, Y., Shang, Y., Zhang, Y., Zong, Z., Feng, J., Yuan, Y., Su, H., Li, N., Sukiennik, N., et al. (2025). Understanding World or Predicting Future? A Comprehensive Survey of World Models. *ACM Computing Surveys*, 58(3), pp. 1–38. Available at: https://arxiv.org/abs/2411.14499 (Accessed: 23 February 2026).

3. He, X., Peng, C., Liu, Z., Wang, B., Zhang, Y., Cui, Q., Kang, F., Jiang, B., An, M., Ren, Y., et al. (2025). Matrix-Game 2.0: An Open-Source, Real-Time, and Streaming Interactive World Model. arXiv preprint arXiv:2508.13009. Available at: https://arxiv.org/abs/2508.13009 (Accessed: 23 February 2026).

4. Lin, D. (2025). A Technical and Practical Comparison of Traditional and Neural Rendering Methods in Games. *Transactions on Computer Science and Intelligent Systems Research*, 9, pp. 264–270. DOI: 10.62051/rkr5g596. Available at: https://doi.org/10.62051/rkr5g596 (Accessed: 23 February 2026).

5. Zeng, W., Li, X., Feng, R., Liu, Z., An, F. and Zhao, J. (2026). Scalable Generative Game Engine: Breaking the Resolution Wall via Hardware-Algorithm Co-Design. arXiv preprint arXiv:2602.00608. Available at: https://arxiv.org/abs/2602.00608 (Accessed: 23 February 2026).

6. Wu, R., He, X., Cheng, M., Yang, T., Zhang, Y., Kang, Z., Cai, X., Wei, X., Guo, C., Li, C. and Cheng, M.-M. (2026). Infinite-World: Scaling Interactive World Models to 1000-Frame Horizons via Pose-Free Hierarchical Memory. arXiv preprint arXiv:2602.02393. Available at: https://arxiv.org/abs/2602.02393 (Accessed: 23 February 2026).

7. Koh, W., Han, S., Lee, S., Yun, S.-Y. and Shin, J. (2026). Generative Visual Code Mobile World Models. arXiv preprint arXiv:2602.01576. Available at: https://arxiv.org/abs/2602.01576 (Accessed: 23 February 2026).

8. Wellington, P. (2026). Growth Equity Update – Edition 47. *Rothschild & Co Equity Capital Markets*, February 2026. Available at: https://www.rothschildandco.com/en/newsroom/insights/2026/02/ga_growth_equity_update_edition_47/ (Accessed: 23 February 2026).

9. Campbell, T. (2026). Goldman Sachs sends software stock warning amid bounce. *TheStreet*, 7 February. Available at: https://www.thestreet.com/investing/stocks/goldman-sachs-sign (Accessed: 23 February 2026).

A Ethics Compliance Form

[Include signed ethics compliance form]

B Project Proposal

[Include original project proposal]

C Game Design Document

[Include full GDD as appendix]